A PRACTICAL, IMPLEMENTATION-FOCUSED GUIDE

MERN
Full-Stack Development Handbook

From an Empty Folder to a Production-Ready Web Application

React • **Node.js**
Express • **MongoDB**

Architecture · Authentication · CRUD · Security · Deployment

Building the Task Management System — a complete end-to-end MERN project

This handbook assumes working knowledge of core JavaScript.
It focuses exclusively on MERN application architecture, implementation, and production readiness.
It does not cover JavaScript fundamentals, Git/GitHub usage, or interview preparation.

Contents

CHAPTER 1

How a MERN Application Works

Before writing a single line of code, you need a mental model of what actually happens between a user clicking something in the browser and data changing in the database. Every feature you build for the rest of this book is just a variation of the same round trip.

1.1 The Full Request Lifecycle

A MERN stack request always travels through the same set of layers, in the same order, whether it is a login form or a task creation button.

MERN Request Lifecycle

```
User
 |  clicks / types / submits
 v
React Component  (JSX + event handler)
 |  calls
 v
Event Handler  (onClick / onSubmit)
 |  calls
 v
API function  (services/taskService.js)
 |  calls
 v
Axios  (src/api/axios.js instance)
 |  sends
 v
HTTP Request  (method + URL + headers + JSON body)
 |
 v
Express App  (app.js / server.js)
 |  matches
 v
Router  (routes/taskRoutes.js)
 |  passes through
 v
Middleware  (auth check, validation, etc.)
 |  calls
 v
Controller  (controllers/taskController.js)
 |  calls
 v
Service / Model layer  (business logic)
 |  calls
 v
```

```
Mongoose  (Task.find / Task.create / ...)
 |  queries
 v
MongoDB  (actual data on disk)
 |  returns
 v
Database Response  (documents / write result)
 |  bubbles back up through
 v
Controller  (shapes { success, data })
 |  sends
 v
JSON Response  (HTTP status + body)
 |  received by
 v
Axios  (resolves the promise / rejects on error)
 |  updates
 v
React State  (useState / context)
 |  triggers
 v
UI Update  (re-render with new data)
```

Each stage has one job:

- **React Component** – renders UI and listens for user interaction.
- **Event Handler** – captures the interaction and decides what to call.
- **API function** – a small wrapper function that knows the exact endpoint and payload shape.
- **Axios** – the HTTP client; attaches base URL, headers, and credentials.
- **Express App / Router** – receives the raw HTTP request and matches it to a route.
- **Middleware** – runs cross-cutting checks (authentication, validation, logging) before the controller.
- **Controller** – orchestrates the request: validates input, calls the model, builds the response.
- **Mongoose** – translates JavaScript calls into MongoDB queries and back into JS objects.
- **MongoDB** – persists and retrieves the actual documents.
- **JSON Response** – a consistent `{ success, data }` envelope sent back to the client.
- **React State** – the single source of truth that drives what the UI shows.

Every chapter in this book is really just teaching you how to implement one segment of this diagram correctly and securely. Keep this chain in your head – it does not change as the app grows.

1.2 Concrete Example: ``User Clicks Create Task''

Let's trace one real interaction end to end in our running example, the **Task Management System**.

Example: Create Task

```
1. User fills "Title" + "Description", clicks "Create Task"
2. TaskForm.jsx  -> handleSubmit(e) fires
3. taskService.js -> createTask({ title, description })
4. axios instance -> POST /api/tasks  (JSON body, withCredentials)
5. Express router  -> router.post("/tasks", protect, createTask)
6. protect middleware -> verifies JWT cookie, attaches req.user
7. createTask controller -> validates body, calls Task.create({...})
8. Mongoose -> inserts document into "tasks" collection
9. MongoDB -> returns the new document with _id
10. Controller -> res.status(201).json({ success: true, data: task })
11. Axios -> promise resolves with response.data
12. React (setTasks) -> new task appended to state array
13. UI -> TaskList re-renders, new TaskCard appears
```

1. **User action:** the user types into two inputs and clicks submit – no network activity yet.
2. **Component:** `TaskForm.jsx` owns the form state via `useState` and calls its `onSubmit` prop.
3. **Service function:** `taskService.js` exports `createTask()`, the only place that knows the endpoint URL.
4. **Axios:** the shared instance adds `baseURL`, `Content-Type`, and cookies automatically.
5. **Express router:** `/api/tasks` maps to the `createTask` controller through the task router.
6. **Middleware:** `protect` rejects the request with 401 if there is no valid session, before any DB access happens.
7. **Controller:** validates the payload and delegates persistence to Mongoose.
8. **Mongoose / MongoDB:** the model writes a new document and returns it with a generated `_id`.
9. **Response:** the controller always replies with the same `{ success, data }` shape (more on this in Chapter 8).
10. **Frontend update:** the resolved promise's data is pushed into React state, and the list re-renders automatically.

If a bug shows up anywhere in a MERN app, walk this chain step by step – component, service, axios, router, middleware, controller, model, database – and check each link. 90% of bugs are a mismatch between two adjacent links (wrong URL, wrong field name, missing `await`, wrong HTTP method).

CHAPTER 2

Creating a MERN Project

A MERN project is really two independent Node.js projects living side by side: a Vite-powered React frontend and an Express backend. They talk to each other only over HTTP, so they can be created, run, and deployed independently.

Project Layout

```
my-app/
|-- frontend/   (Vite + React, runs on :5173)
|-- backend/    (Express + MongoDB, runs on :5000)
```

2.1 Scaffolding the Frontend

```
mkdir my-app && cd my-app

npm create vite@latest frontend -- --template react
cd frontend
npm install
npm run dev
```

This gives you a working React app with hot module reload. We will restructure `src/` into a proper folder layout in Chapter 3, and add the Axios API layer in Chapter 7.

2.2 Scaffolding the Backend

```
cd my-app
mkdir backend && cd backend
npm init -y
```

Install the packages every MERN backend in this book relies on:

```
npm install express mongoose dotenv cors cookie-parser bcryptjs jsonwebtoken
npm install -D nodemon
```

A minimal entry point to confirm everything is wired correctly:

```
// backend/server.js
const express = require("express");
const cors = require("cors");
const cookieParser = require("cookie-parser");
require("dotenv").config();

const app = express();

app.use(cors({ origin: process.env.CLIENT_URL, credentials: true }));
app.use(express.json());
app.use(cookieParser());
```

Package	Purpose
`express`	The HTTP server framework – routing, middleware, request/response handling.
`mongoose`	ODM (Object Document Mapper) that maps JS schemas/models to MongoDB collections.
`dotenv`	Loads environment variables (`PORT`, `MONGO_URI`, `JWT_SECRET`) from a `.env` file.
`cors`	Enables Cross-Origin Resource Sharing so the frontend origin can call the API.
`cookie-parser`	Parses the `Cookie` header so the server can read the JWT stored in an HTTP-only cookie.
`bcryptjs`	Hashes and compares passwords – never store plain-text passwords.
`jsonwebtoken`	Issues and verifies JWTs used for stateless authentication.
`nodemon` (dev)	Restarts the server automatically on file changes during development.

```
app.get("/api/health", (req, res) => {
  res.json({ success: true, message: "API is running" });
});

const PORT = process.env.PORT || 5000;
app.listen(PORT, () => console.log(`Server running on port ${PORT}`));
```

`MONGO_URI`, `JWT_SECRET`, `PORT`, and `CLIENT_URL` belong in a `.env` file that is git-ignored. Database connection setup with Mongoose is covered in Chapter 9.

Add `"dev": "nodemon server.js"` and `"start": "node server.js"` to `backend/package.json` scripts now – you will use `npm run dev` constantly for the rest of this book.

CHAPTER 3

React Project Structure

A flat `src/` folder works for a to-do demo. It falls apart the moment your app has auth, protected routes, and more than three pages. Here is the structure used throughout this book for the Task Management System.

frontend/src Structure

```
frontend/src/
|-- api/
|   `-- axios.js          (single configured axios instance)
|-- services/
|   |-- authService.js    (login, register, logout)
|   `-- taskService.js    (getTasks, createTask, updateTask...)
|-- components/
|   |-- TaskCard.jsx
|   |-- TaskList.jsx
|   `-- Navbar.jsx
|-- pages/
|   |-- Dashboard.jsx
|   |-- Login.jsx
|   `-- TaskDetails.jsx
|-- layouts/
|   `-- MainLayout.jsx    (Navbar + Sidebar + <Outlet/>)
|-- hooks/
|   `-- useAuth.js
|-- context/
|   `-- AuthContext.jsx
|-- utils/
|   `-- formatDate.js
|-- assets/
|   `-- logo.svg
|-- App.jsx
`-- main.jsx
```

3.1 What Goes Where

The distinction between `api/` and `services/` matters: `api/axios.js` knows nothing about "tasks" or "users" – it's generic HTTP plumbing. `services/taskService.js` knows the specific endpoints and payload shapes for tasks. This separation means changing your backend's base URL never requires touching business logic files.

Folder	Contains	Should NOT contain
`components/`	Small, reusable, presentation-focused UI pieces (`TaskCard`, `Button`, `Navbar`)	Route definitions, data fetching logic, page-level layout
`pages/`	Route-level components that compose components into a full screen (`Dashboard`, `Login`)	Low-level reusable UI meant to be shared across pages
`layouts/`	Shell components wrapping multiple pages (header/sidebar + `<Outlet/>`)	Business logic or data fetching for a specific feature
`hooks/`	Custom hooks encapsulating reusable stateful logic (`useAuth`, `useDebounce`)	JSX markup / rendered UI
`context/`	React Context providers and their state (`AuthContext`, `ThemeContext`)	One-off local component state that doesn't need to be global
`services/`	Functions that call the `api/` layer for a specific resource (`taskService.getTasks`)	Raw axios configuration, JSX
`api/`	The axios instance itself: base URL, interceptors, headers	Business logic specific to one resource
`utils/`	Pure helper functions with no React or network dependency (`formatDate`, `truncate`)	Components, hooks, anything using `useState`

A quick test for "does this belong in `components/` or `pages/`?": if it needs its own route in `App.jsx`, it's a page. If it's used inside two or more pages, it's a component.

CHAPTER 4

Components and Pages

4.1 Component Tree

A realistic component tree for the dashboard screen of the Task Management System looks like this:

Task Manager Component Tree

```
App
|-- Navbar
|-- MainLayout
|   |-- Sidebar
|   `-- <Outlet/>  (Routes render here)
|       |-- Dashboard (page)
|       |   |-- StatsCard  x3   (Total / Pending / Done)
|       |   `-- TaskList
|       |       `-- TaskCard  (one per task)
|       `-- Profile (page)
```

Reusable components (`StatsCard`, `TaskCard`, `TaskList`) know nothing about routing or data fetching – they receive data via props and render it. **Page components** (`Dashboard`, `Profile`) own the data fetching and pass slices of it down. **Layouts** (`MainLayout`) provide the persistent chrome (nav, sidebar) around whichever page is active.

4.2 A Reusable Component

```jsx
// components/TaskCard.jsx
export default function TaskCard({ task, onStatusChange }) {
  const statusColor = {
    pending: "gray",
    "in-progress": "orange",
    done: "green",
  }[task.status];

  return (
    <div className="task-card">
      <h3>{task.title}</h3>
      <p>{task.description}</p>
      <span className={`badge badge-${statusColor}`}>{task.status}</span>

      {task.status !== "done" && (
        <button onClick={() => onStatusChange(task._id, "done")}>
          Mark Done
        </button>
      )}
    </div>
  );
}
```

Note the pattern: `TaskCard` receives `task` and a callback `onStatusChange` as props. It never calls the API itself – that keeps it reusable and easy to test in isolation.

4.3 A Page Component

```jsx
// pages/Dashboard.jsx
import { useEffect, useState } from "react";
import TaskList from "../components/TaskList";
import StatsCard from "../components/StatsCard";
import { getTasks, updateTaskStatus } from "../services/taskService";

export default function Dashboard() {
  const [tasks, setTasks] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    getTasks()
      .then((data) => setTasks(data))
      .finally(() => setLoading(false));
  }, []);

  const handleStatusChange = async (id, status) => {
    const updated = await updateTaskStatus(id, status);
    setTasks((prev) => prev.map((t) => (t._id === id ? updated : t)));
  };

  if (loading) return <p>Loading tasks...</p>;

  const pending = tasks.filter((t) => t.status === "pending").length;
  const done = tasks.filter((t) => t.status === "done").length;

  return (
    <div className="dashboard">
      <div className="stats-row">
        <StatsCard label="Total" value={tasks.length} />
        <StatsCard label="Pending" value={pending} />
        <StatsCard label="Done" value={done} />
      </div>
      <TaskList tasks={tasks} onStatusChange={handleStatusChange} />
    </div>
  );
}
```

`Dashboard` owns the state (`tasks`, `loading`) and the data fetching, then hands everything down as props. This is the standard "smart page, dumb component" split used throughout the book.

4.4 Forms and Conditional Rendering

```jsx
// components/TaskForm.jsx
import { useState } from "react";

export default function TaskForm({ onCreate }) {
  const [form, setForm] = useState({ title: "", description: "" });
  const [error, setError] = useState("");

  const handleChange = (e) =>
```

```
    setForm({ ...form, [e.target.name]: e.target.value });

  const handleSubmit = async (e) => {
    e.preventDefault();
    if (!form.title.trim()) return setError("Title is required");

    await onCreate(form);
    setForm({ title: "", description: "" });
    setError("");
  };

  return (
    <form onSubmit={handleSubmit}>
      <input name="title" value={form.title} onChange={handleChange} />
      <textarea
        name="description"
        value={form.description}
        onChange={handleChange}
      />
      {error && <p className="error">{error}</p>}
      <button type="submit">Create Task</button>
    </form>
  );
}
```

Use the spread operator (`...form`) to update one field of an object in state without losing the others, and computed keys (`[e.target.name]`) to reuse a single `handleChange` for every input in the form.

CHAPTER 5

Routing with React Router

5.1 Core Building Blocks

```jsx
// App.jsx
import { BrowserRouter, Routes, Route } from "react-router-dom";
import MainLayout from "./layouts/MainLayout";
import ProtectedRoute from "./components/ProtectedRoute";
import Login from "./pages/Login";
import Register from "./pages/Register";
import Dashboard from "./pages/Dashboard";
import TaskDetails from "./pages/TaskDetails";
import Profile from "./pages/Profile";

export default function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/login" element={<Login />} />
        <Route path="/register" element={<Register />} />

        <Route
          element={
            <ProtectedRoute>
              <MainLayout />
            </ProtectedRoute>
          }
        >
          <Route path="/dashboard" element={<Dashboard />} />
          <Route path="/tasks/:id" element={<TaskDetails />} />
          <Route path="/profile" element={<Profile />} />
        </Route>
      </Routes>
    </BrowserRouter>
  );
}
```

- `BrowserRouter` – wraps the app and enables client-side routing via the History API.
- `Routes` / `Route` – declarative path-to-component mapping.
- `Link` / `NavLink` – render `<a>` tags without a full page reload; `NavLink` adds an `active` class automatically.
- `useNavigate` – programmatic navigation (e.g. redirect after login).
- `useParams` – reads dynamic segments like `:id` from the URL.
- Nesting a `Route` without a `path` under a parent that renders `<Outlet/>` creates a **layout route** – every child shares the same shell.

```jsx
// layouts/MainLayout.jsx
```

```jsx
import { Outlet } from "react-router-dom";
import Navbar from "../components/Navbar";
import Sidebar from "../components/Sidebar";

export default function MainLayout() {
  return (
    <div className="app-shell">
      <Navbar />
      <div className="app-body">
        <Sidebar />
        <main><Outlet /></main>
      </div>
    </div>
  );
}
```

```jsx
// pages/TaskDetails.jsx
import { useParams, useNavigate } from "react-router-dom";

export default function TaskDetails() {
  const { id } = useParams(); // "tasks/64f..." -> id = "64f..."
  const navigate = useNavigate();

  const handleBack = () => navigate("/dashboard");
  // ...fetch task by id, render details
}
```

5.2 Protected Routes

Protected routes stop unauthenticated users from ever rendering a page that needs a logged-in user – the check happens before the page component mounts.

Protected Route Flow

```
Browser requests /dashboard
        |
        v
<ProtectedRoute> renders
        |
        v
useAuth() -> is user logged in?
        |
    +---+---+
    |       |
    NO      YES
    |       |
    v       v
<Navigate  render children
 to="/login"/>   (<MainLayout/> -> <Outlet/>)
```

```jsx
// components/ProtectedRoute.jsx
import { Navigate } from "react-router-dom";
import { useAuth } from "../hooks/useAuth";
```

```
export default function ProtectedRoute({ children }) {
  const { user, loading } = useAuth();

  if (loading) return <p>Checking session...</p>;
  if (!user) return <Navigate to="/login" replace />;

  return children;
}
```

`replace` prevents the login page from being pushed onto the history stack again, so the browser's back button doesn't bounce the user between `/login` and the page that redirected them.

A client-side `ProtectedRoute` only hides UI. It is **not** a security boundary – the Express backend must independently verify the JWT on every protected endpoint (Chapter 5's frontend check and the backend's `protect` middleware from Chapter 2 are two separate, both-required layers).

CHAPTER 6

React State Management

6.1 The Core Hooks in Practice

```
// useState – local, per-render state
const [tasks, setTasks] = useState([]);

// useEffect – side effects (fetching, subscriptions, timers)
useEffect(() => {
  fetchTasks();
}, []); // empty array = run once, on mount

// useContext – read shared state without prop drilling
const { user, logout } = useContext(AuthContext);

// useRef – mutable value that survives re-renders, no re-render on change
const inputRef = useRef(null);
useEffect(() => { inputRef.current?.focus(); }, []);

// useMemo – cache an expensive computed value
const doneCount = useMemo(
  () => tasks.filter((t) => t.status === "done").length,
  [tasks]
);

// useCallback – cache a function identity across re-renders
const handleDelete = useCallback(
  (id) => setTasks((prev) => prev.filter((t) => t._id !== id)),
  []
);
```

6.2 A Custom Hook: useAuth

```
// hooks/useAuth.js
import { useContext } from "react";
import { AuthContext } from "../context/AuthContext";

export function useAuth() {
  const context = useContext(AuthContext);
  if (!context) {
    throw new Error("useAuth must be used within an AuthProvider");
  }
  return context; // { user, loading, login, logout }
}
```

Custom hooks let you extract stateful logic (auth status, form handling, debounced search) into a reusable function without duplicating `useEffect`/`useState` wiring across components.

6.3 Common Mistakes: Infinite API Call Loops

The single most common MERN bug is an effect that re-triggers itself forever.

```
// BUG: no dependency array -> runs after every render,
// and setTasks triggers another render -> infinite loop
function Dashboard() {
  const [tasks, setTasks] = useState([]);

  useEffect(() => {
    getTasks().then(setTasks);
  }); // <-- missing [] !!

  return <TaskList tasks={tasks} />;
}
```

Every call to `setTasks` causes a re-render, which runs the effect again (no dependency array means "run after every render"), which calls `setTasks` again – forever. This shows up as a browser tab hammering your API with hundreds of requests per second.

Fix: add the dependency array, even if empty.

```
useEffect(() => {
  getTasks().then(setTasks);
}, []); // runs once, on mount
```

A second, subtler version of the same bug happens with objects or functions in the dependency array:

```
// BUG: filters is a new object every render -> effect
// "sees" a new dependency every time -> runs every render
function TaskList() {
  const filters = { status: "pending" }; // new reference each render

  useEffect(() => {
    getTasks(filters).then(setTasks);
  }, [filters]); // filters is never === the previous filters
}
```

Fix: memoize the object, or depend on its primitive fields instead.

```
const filters = useMemo(() => ({ status }), [status]);

useEffect(() => {
  getTasks(filters).then(setTasks);
}, [filters]);

// or simply:
useEffect(() => {
  getTasks({ status }).then(setTasks);
}, [status]); // depend on the primitive, not the wrapper object
```

Rule of thumb: every value read inside `useEffect` that can change over time belongs in the dependency array. If adding it causes a loop, the real fix is almost always to memoize that value or depend on a primitive derived from it – not to remove it from the array.

CHAPTER 7

Axios and the Frontend API Layer

7.1 A Centralized Axios Instance

Every service file in the app should import one shared, pre-configured axios instance rather than calling `axios.get(...)` directly with a hardcoded URL.

```
// src/api/axios.js
import axios from "axios";

const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL, // e.g. http://localhost:5000/api
  withCredentials: true, // send the JWT cookie automatically
  headers: { "Content-Type": "application/json" },
});

export default api;
```

- **baseURL** – every call to `api.get("/tasks")` actually hits `{VITE_API_URL}/tasks`, so the URL never needs to be repeated.
- **headers** – default headers applied to every request; individual calls can still override them.
- **withCredentials** – required for cookie-based JWT auth (Chapter 14) so the browser attaches the HTTP-only auth cookie on cross-origin requests.

`import.meta.env.VITE_API_URL` comes from Vite's env system. Define it in `frontend/.env` as `VITE_API_URL=http://localhost:5000/api` – Vite only exposes variables prefixed with `VITE_` to client code.

7.2 Interceptors

Interceptors let you run code on *every* request or response without repeating it in each service function.

```
// src/api/axios.js (continued)
import { store } from "../store"; // or however you track loading globally

api.interceptors.request.use((config) => {
  store.setLoading(true);
  return config;
});

api.interceptors.response.use(
  (response) => {
    store.setLoading(false);
    return response;
  },
```

```
  (error) => {
    store.setLoading(false);

    if (error.response?.status === 401) {
      window.location.href = "/login"; // session expired or invalid
    }

    return Promise.reject(error);
  }
);
```

This pattern gives every screen a global loading indicator and a single place that handles "you got logged out" – individual components don't need to check for 401 themselves.

7.3 Error Handling in Service Functions

```
// services/taskService.js
import api from "../api/axios";

export async function getTasks() {
  try {
    const res = await api.get("/tasks");
    return res.data.data; // unwrap { success, data }
  } catch (err) {
    const message = err.response?.data?.message || "Failed to load tasks";
    throw new Error(message);
  }
}
```

```
// pages/Dashboard.jsx (usage)
useEffect(() => {
  getTasks()
    .then(setTasks)
    .catch((err) => setError(err.message));
}, []);
```

Keep `try/catch` inside the service function, not scattered across every component. Components should only need to handle the friendly error message that the service throws, not raw axios error objects.

Don't swallow errors silently (`catch(() => {})`). At minimum, surface a message in the UI or log it – a request that silently fails is far harder to debug than one that throws visibly.

CHAPTER 8

Establishing API Communication

This chapter connects everything from Chapters 1–7 into one fully working round trip: a React page calling `api.get("/tasks")` and rendering whatever MongoDB returns. Read this chapter slowly – every CRUD feature for the rest of the book is a variation of this exact pattern.

8.1 The Complete Cycle, Labeled

GET /api/tasks -- Full Cycle

```
React: Dashboard.jsx
  useEffect(() => { taskService.getTasks() }, [])
       |
       v
services/taskService.js
  api.get("/tasks")
       |
       v
Axios instance (src/api/axios.js)
  + baseURL:  http://localhost:5000/api
  + withCredentials: true (sends JWT cookie)
       |
       v
HTTP Request  ------------------------------>
  GET /api/tasks
  Headers: Cookie: token=<jwt>; Content-Type: application/json
  Body: (none - GET has no body)
       |
       v
Express app.js
  app.use("/api/tasks", taskRoutes)
       |
       v
routes/taskRoutes.js
  router.get("/", protect, getTasks)
       |
       v
middleware/auth.js  (protect)
  verifies JWT from cookie -> attaches req.user
       |
       v
controllers/taskController.js
  getTasks(req, res)
       |
       v
```

```
  Mongoose
    Task.find({ user: req.user._id })
          |
          v
  MongoDB
    scans "tasks" collection, returns matching documents
          |
          v
  controllers/taskController.js
    res.status(200).json({ success: true, data: tasks })
          |
          v
  HTTP Response  <------------------------------
    Status: 200 OK
    Body: { "success": true, "data": [ {...}, {...} ] }
          |
          v
  Axios instance
    resolves promise with response.data
          |
          v
  services/taskService.js
    returns res.data.data  (the array of tasks)
          |
          v
  React: Dashboard.jsx
    setTasks(data) -> re-render -> TaskList shows TaskCards
```

8.2 The Frontend Side

```jsx
// src/services/taskService.js
import api from "../api/axios";

export async function getTasks() {
  try {
    const res = await api.get("/tasks");
    return res.data.data;
  } catch (err) {
    throw new Error(err.response?.data?.message || "Failed to load tasks");
  }
}
```

```jsx
// src/pages/Dashboard.jsx
import { useEffect, useState } from "react";
import { getTasks } from "../services/taskService";
import TaskList from "../components/TaskList";

export default function Dashboard() {
  const [tasks, setTasks] = useState([]);
  const [error, setError] = useState("");

  useEffect(() => {
    getTasks()
```

```
      .then(setTasks)
      .catch((err) => setError(err.message));
  }, []);

  if (error) return <p className="error">{error}</p>;
  return <TaskList tasks={tasks} />;
}
```

8.3 The Backend Side

```
// routes/taskRoutes.js
const express = require("express");
const router = express.Router();
const { protect } = require("../middleware/auth");
const { getTasks, createTask } = require("../controllers/taskController");

router.get("/", protect, getTasks);
router.post("/", protect, createTask);

module.exports = router;
```

```
// controllers/taskController.js
const Task = require("../models/Task");

exports.getTasks = async (req, res) => {
  try {
    const tasks = await Task.find({ user: req.user._id }).sort({ createdAt: -1 });

    res.status(200).json({
      success: true,
      data: tasks,
    });
  } catch (err) {
    res.status(500).json({
      success: false,
      message: "Failed to fetch tasks",
    });
  }
};
```

```
// app.js (wiring)
const express = require("express");
const app = express();

app.use(express.json());
app.use("/api/tasks", require("./routes/taskRoutes"));
app.use("/api/auth", require("./routes/authRoutes"));

module.exports = app;
```

Notice the controller never talks to Express's `req`/`res` objects inside Mongoose calls, and never talks to MongoDB directly inside routing code. Each layer only knows about the layer imme-

diately next to it – that's what makes this chain easy to debug and to test.

8.4 Standard Response Shapes

To keep the frontend's unwrapping logic (`res.data.data`) consistent everywhere, every endpoint in this book returns one of the following shapes.

Shape	Used for	Example
`{ success, data }`	Single resource or list fetch/create/update	`GET /tasks`, `POST /tasks`
`{ success, message }`	Action with no payload to return	`DELETE /tasks/:id`, `POST /auth/logout`
`{ success, data, page, limit, total, totalPages }`	Paginated list endpoints (Chapter 20)	`GET /tasks?page=2&limit=10`

Never mix shapes for the same endpoint across success and error cases – e.g. returning `data` on success but a bare string on failure. Always keep `success: boolean` as the first field so the frontend can branch on it consistently regardless of HTTP status code edge cases.

When a new feature isn't working, check these five things in order: (1) is the URL in `taskService.js` correct, (2) is the HTTP method correct, (3) does the Express route match that method and path, (4) does the controller query the right field, (5) does the response shape match what the frontend expects. This covers the overwhelming majority of "it's just not working" bugs in a MERN app.

CHAPTER 9

Express Server Setup

9.1 Why Split `app.js` and `server.js`

WHAT: A production-grade Express backend separates the **application definition** (`app.js`) from the **server bootstrap** (`server.js`).

WHY: `app.js` defines what the application *is* — middleware, routes, error handlers — with zero knowledge of ports, sockets, or the database. `server.js` defines how the application *runs* — it starts the HTTP listener and connects to MongoDB. This separation makes the app testable (you can import `app` into a test file with tools like Supertest without ever binding a port) and keeps startup concerns out of business logic.

HOW: Export the configured Express instance from `app.js`. Import it in `server.js`, connect to the database, then call `app.listen(PORT)`.

9.1.1 backend/src/app.js

```
const express = require("express");
const cors = require("cors");
const cookieParser = require("cookie-parser");

const authRoutes = require("./routes/auth.routes");
const userRoutes = require("./routes/user.routes");
const taskRoutes = require("./routes/task.routes");

const app = express();

// ---- Core middleware ----
app.use(express.json()); // parses JSON request bodies -> req.body
app.use(express.urlencoded({ extended: true }));
app.use(
  cors({
    origin: process.env.CLIENT_URL || "http://localhost:5173",
    credentials: true, // allow cookies to cross origin
  })
);
app.use(cookieParser()); // parses cookies -> req.cookies

// ---- Health check ----
app.get("/api/health", (req, res) => {
  res.status(200).json({ status: "ok" });
});

// ---- Routes ----
app.use("/api/auth", authRoutes);
app.use("/api/users", userRoutes);
app.use("/api/tasks", taskRoutes);

// ---- 404 handler ----
```

```
app.use((req, res) => {
  res.status(404).json({ message: "Route not found" });
});

// ---- Centralized error handler ----
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(err.statusCode || 500).json({
    message: err.message || "Internal Server Error",
  });
});

module.exports = app;
```

9.1.2 backend/src/server.js

```
require("dotenv").config();
const app = require("./app");
const connectDB = require("./config/db");

const PORT = process.env.PORT || 5000;

const startServer = async () => {
  try {
    await connectDB();
    app.listen(PORT, () => {
      console.log(`Server running on port ${PORT}`);
    });
  } catch (error) {
    console.error("Failed to start server:", error.message);
    process.exit(1);
  }
};

startServer();
```

Startup Sequence

```
node server.js
      |
      v
load .env (dotenv)
      |
      v
require app.js  ---> builds Express app + middleware + routes (no side effects)
      |
      v
connectDB()  ------> await mongoose.connect(MONGO_URI)
      |
      v
app.listen(PORT) --> HTTP server starts accepting requests
```

9.2 Core Concepts Recap

- **app** — the Express application object; a callable request handler you configure once.
- **Middleware** — functions of shape `(req, res, next)` that run in order for every matching request; they can inspect/modify `req`/`res` or short-circuit with a response.
- **Routes** — map an HTTP method + path to a handler function.
- **req (Request)** — incoming data: `req.body`, `req.params`, `req.query`, `req.cookies`, `req.headers`.
- **res (Response)** — outgoing data: `res.status()`, `res.json()`, `res.cookie()`, `res.send()`.

Middleware order matters. `express.json()` must run *before* any route that reads `req.body`, otherwise the body will be `undefined`.

Never call `app.listen()` inside `app.js`. Doing so binds a port as a side effect of importing the module, which breaks unit testing and can cause duplicate listeners if the module is required twice.

CHAPTER 10

Express Routing

10.1 HTTP Methods and Their Purpose

- **GET** — retrieve a resource or collection; no body, safe and idempotent.
- **POST** — create a new resource; carries a request body.
- **PUT** — replace a resource entirely (send the full object).
- **PATCH** — partially update a resource (send only changed fields).
- **DELETE** — remove a resource.

In practice many REST APIs use `PUT` where `PATCH` would be more semantically correct. This handbook uses `PUT` for task updates since the frontend form always sends the full task object.

10.2 Route Files with express.Router()

WHAT: `express.Router()` creates a mini, mountable Express application scoped to one resource.

WHY: Without it, every route would live in one giant `app.js`, making the file unmanageable as the API grows.

HOW: Define routes in `routes/*.routes.js`, export the router, and mount it in `app.js` with `app.use("/api/tasks", taskRoutes)`.

10.2.1 routes/auth.routes.js

```
const express = require("express");
const { register, login, logout, getMe } = require("../controllers/auth.controller");
const { protect } = require("../middleware/auth.middleware");

const router = express.Router();

router.post("/register", register);
router.post("/login", login);
router.post("/logout", logout);
router.get("/me", protect, getMe);

module.exports = router;
```

10.2.2 routes/user.routes.js

```
const express = require("express");
const { getUsers, getUserById, updateUser } = require("../controllers/user.controller
    ");
const { protect } = require("../middleware/auth.middleware");

const router = express.Router();

router.get("/", protect, getUsers);
router.get("/:id", protect, getUserById);
router.put("/:id", protect, updateUser);

module.exports = router;
```

10.2.3 routes/task.routes.js

```
const express = require("express");
const {
  createTask,
  getTasks,
  getTaskById,
  updateTask,
  deleteTask,
} = require("../controllers/task.controller");
const { protect } = require("../middleware/auth.middleware");

const router = express.Router();

router.use(protect); // every task route requires authentication

router.post("/", createTask);
router.get("/", getTasks);
router.get("/:id", getTaskById);
router.put("/:id", updateTask);
router.delete("/:id", deleteTask);

module.exports = router;
```

10.2.4 routes/profile.routes.js

```
const express = require("express");
const { getProfile, updateProfile } = require("../controllers/profile.controller");
const { protect } = require("../middleware/auth.middleware");

const router = express.Router();

router.get("/", protect, getProfile);
router.put("/", protect, updateProfile);

module.exports = router;
```

10.3 Route Params vs Query Params vs Body

- **Route params** — part of the URL path, used to identify a specific resource. `/tasks/:id` → `req.params.id`.
- **Query params** — key/value pairs after `?`, used for filtering, pagination, sorting. `/tasks?page=1&limit=10` → `req.query.page`, `req.query.limit`.
- **Request body** — JSON payload sent with POST/PUT/PATCH, used to create or update data. `req.body` is only populated because `express.json()` middleware ran first.

```
// GET /api/tasks?page=2&limit=10&status=todo
const getTasks = async (req, res) => {
  const { page = 1, limit = 10, status } = req.query;
  // ...build filter and pagination from query params
};

// GET /api/tasks/:id
const getTaskById = async (req, res) => {
  const { id } = req.params; // route param identifies the exact task
};
```

10.4 Task Routes Reference

Method	Path	Purpose
POST	/api/tasks	Create a new task for the logged-in user
GET	/api/tasks	List tasks (supports pagination & filtering via query params)
GET	/api/tasks/:id	Fetch a single task by its ObjectId
PUT	/api/tasks/:id	Replace/update an existing task
DELETE	/api/tasks/:id	Remove a task

Keep route files thin. They should only wire paths to controller functions and attach middleware — no business logic, no direct database calls.

CHAPTER 11

MVC + Service Layer Architecture

11.1 Professional Backend Folder Structure

backend/src Folder Tree

```
backend/src/
|-- config/
|   `-- db.js                MongoDB connection setup
|-- controllers/
|   |-- auth.controller.js   HTTP layer: parses req, calls service, sends res
|   `-- task.controller.js
|-- middleware/
|   |-- auth.middleware.js   protect(), role checks
|   `-- error.middleware.js  centralized error handler
|-- models/
|   |-- User.js              Mongoose schema/model
|   `-- Task.js
|-- routes/
|   |-- auth.routes.js       path -> controller wiring
|   `-- task.routes.js
|-- services/
|   |-- auth.service.js      business logic + Mongoose queries
|   `-- task.service.js
|-- validators/
|   `-- task.validator.js    input validation schemas
|-- utils/
|   |-- generateToken.js
|   `-- asyncHandler.js
|-- app.js                   Express app + middleware + routes
`-- server.js                HTTP listener + DB connection
```

11.2 The Request Flow

Layered Request Flow

```
Client Request
     |
     v
  Route          (path matching: POST /api/tasks)
     |
     v
 Middleware       (auth check, validation)
     |
     v
```

```
 Controller       (parse req, call service, shape res)
     |
     v
  Service         (business rules, orchestrates model calls)
     |
     v
   Model          (Mongoose schema methods: Task.create, Task.find)
     |
     v
MongoDB           (actual document storage)
```

11.3 Anti-Pattern: Everything in the Route Handler

The following is what an unstructured, "just make it work" route looks like. It mixes validation, auth, database access, and business logic in one function. This does not scale past a handful of endpoints.

```
// routes/task.routes.js -- DO NOT DO THIS
router.post("/", async (req, res) => {
  // auth check inline
  const token = req.cookies.token;
  if (!token) return res.status(401).json({ message: "Not authenticated" });
  let userId;
  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    userId = decoded.id;
  } catch (e) {
    return res.status(401).json({ message: "Invalid token" });
  }

  // validation inline
  const { title, description, status, priority, dueDate } = req.body;
  if (!title || title.trim() === "") {
    return res.status(400).json({ message: "Title is required" });
  }
  if (status && !["todo", "progress", "completed"].includes(status)) {
    return res.status(400).json({ message: "Invalid status" });
  }

  // business logic + db access inline
  try {
    const existingCount = await Task.countDocuments({ user: userId });
    if (existingCount >= 100) {
      return res.status(400).json({ message: "Task limit reached" });
    }
    const task = new Task({ title, description, status, priority, dueDate, user:
        userId });
    await task.save();
    const populated = await Task.findById(task._id).populate("user", "name email");
    return res.status(201).json(populated);
  } catch (err) {
    return res.status(500).json({ message: err.message });
  }
```

```
});
```

11.4 Refactored: Route -> Controller -> Service

11.4.1 routes/task.routes.js

```
const express = require("express");
const { createTask } = require("../controllers/task.controller");
const { protect } = require("../middleware/auth.middleware");
const { validateTask } = require("../validators/task.validator");

const router = express.Router();

router.post("/", protect, validateTask, createTask);

module.exports = router;
```

11.4.2 controllers/task.controller.js

```
const taskService = require("../services/task.service");

// Controller: translates HTTP <-> service calls. No Mongoose here.
const createTask = async (req, res, next) => {
  try {
    const task = await taskService.createTask(req.body, req.user.id);
    res.status(201).json(task);
  } catch (err) {
    next(err); // delegate to centralized error middleware
  }
};

module.exports = { createTask };
```

11.4.3 services/task.service.js

```
const Task = require("../models/Task");

const MAX_TASKS_PER_USER = 100;

// Service: owns business rules and all Mongoose interaction.
const createTask = async (taskData, userId) => {
  const existingCount = await Task.countDocuments({ user: userId });
  if (existingCount >= MAX_TASKS_PER_USER) {
    const err = new Error("Task limit reached");
    err.statusCode = 400;
    throw err;
  }

  const task = await Task.create({ ...taskData, user: userId });
  return task.populate("user", "name email");
};

module.exports = { createTask };
```

11.5 Layer Responsibilities

Layer	Responsibility	Should NOT contain
Route	Map HTTP method+path to a controller; attach middleware	Business logic, database calls, validation logic
Middleware	Cross-cutting concerns: auth, validation, logging, error handling	Resource-specific business rules
Controller	Read `req`, call one service function, shape `res`	Direct Mongoose queries, complex business rules
Service	Business rules, orchestration, calls to one or more models	`req`/`res` objects, HTTP status codes
Model	Schema definition, field-level validation, Mongoose hooks	Business rules that span multiple resources

A good litmus test: if you could reuse the function from a CLI script or a cron job (no `req`/`res` in sight), it belongs in the service layer.

CHAPTER 12

MongoDB Setup

12.1 Core Concepts

- **MongoDB** — a document-oriented NoSQL database; stores data as flexible, JSON-like documents instead of rows in tables.
- **MongoDB Atlas** — MongoDB's official managed cloud hosting service; provisions clusters without you managing servers.
- **Database** — a logical container for collections (e.g. `taskmanager`).
- **Collection** — a group of documents, analogous to a SQL table (e.g. `tasks`, `users`).
- **Document** — a single BSON record inside a collection, analogous to a row (e.g. one task).
- **ObjectId** — MongoDB's default 12-byte unique identifier, auto-assigned to a document's `_id` field.
- **BSON** — Binary JSON; the binary-encoded format MongoDB actually stores documents in on disk (adds types like Date and ObjectId that plain JSON lacks).

MongoDB Hierarchy

```
MongoDB Server / Cluster
        |
        v
    Database            (e.g. "taskmanager")
        |
        v
    Collection           (e.g. "tasks", "users")
        |
        v
    Document             ({ _id, title, status, ... })
        |
        v
    Fields               (title: "Fix bug", status: "todo")
```

12.2 Connecting with Mongoose

WHAT: Mongoose is an ODM (Object Data Modeling) library that sits between your Node.js code and MongoDB, giving you schemas, validation, and query helpers.

WHY: Raw MongoDB driver calls are untyped and repetitive. Mongoose lets you define a schema once and get consistent validation and query building everywhere.

HOW: Create a single connection module (`config/db.js`) that reads the connection string from environment variables and connects once at startup.

12.2.1 config/db.js

```
const mongoose = require("mongoose");

const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGO_URI);
    console.log(`MongoDB connected: ${conn.connection.host}`);
  } catch (error) {
    console.error(`MongoDB connection error: ${error.message}`);
    process.exit(1);
  }
};

module.exports = connectDB;
```

12.2.2 server.js (excerpt)

```
const connectDB = require("./config/db");

connectDB().then(() => {
  app.listen(PORT, () => console.log(`Server running on port ${PORT}`));
});
```

12.3 MongoDB Atlas Connection String

An Atlas connection string follows the `mongodb+srv://` format:

```
// .env
MONGO_URI=mongodb+srv://<username>:<password>@cluster0.mongodb.net/taskmanager?
    retryWrites=true&w=majority
```

Atlas requires you to whitelist IP addresses under Network Access before any connection succeeds; for local development add your current IP (or `0.0.0.0/0` temporarily for testing only).

Never commit `.env` or hardcode the connection string with real credentials into source control. Add `.env` to `.gitignore` and provide a `.env.example` with placeholder values instead.

CHAPTER 13

Mongoose Schemas and Models

13.1 Schema vs Model

WHAT: A **schema** is the blueprint describing a document's shape, field types, and validation rules. A **model** is the compiled, usable interface Mongoose builds from that schema to actually query a collection.

WHY: Separating the two lets you define validation and structure once (schema) while getting a rich query API (model) for free: `find`, `create`, `findById`, `updateOne`, and so on.

HOW: Call `new mongoose.Schema({...})` to define fields, then `mongoose.model("Task", taskSchema)` to compile it into a model bound to the `tasks` collection (Mongoose pluralizes and lowercases the name automatically).

13.2 Task Schema

```
// models/Task.js
const mongoose = require("mongoose");

const taskSchema = new mongoose.Schema(
  {
    title: {
      type: String,
      required: [true, "Title is required"],
      trim: true,
    },
    description: {
      type: String,
      default: "",
    },
    status: {
      type: String,
      enum: ["todo", "progress", "completed"],
      default: "todo",
    },
    priority: {
      type: String,
      enum: ["low", "medium", "high"],
      default: "medium",
    },
    dueDate: {
      type: Date,
    },
    user: {
      type: mongoose.Schema.Types.ObjectId,
      ref: "User",
      required: true,
    },
```

```
  },
  { timestamps: true }
);

const Task = mongoose.model("Task", taskSchema);

module.exports = Task;
```

13.3 Field Options Explained

Option	Meaning
`type`	The data type Mongoose casts and validates against (`String`, `Number`, `Date`, `Boolean`, `ObjectId`, ...).
`required`	Rejects document creation/save if the field is missing; can pair with a custom error message.
`enum`	Restricts a String/Number field to a fixed set of allowed values.
`default`	Value used when the field is not supplied at creation time.
`ref`	Names the model that an `ObjectId` field points to, enabling `.populate()`.
`trim`	Strips leading/trailing whitespace from String values.
`timestamps`	Schema option (not a field option) that auto-adds and maintains `createdAt`/`updatedAt`.

13.4 User Schema (for later chapters)

```
// models/User.js
const mongoose = require("mongoose");

const userSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: [true, "Name is required"],
      trim: true,
    },
    email: {
      type: String,
      required: [true, "Email is required"],
      unique: true,
      lowercase: true,
      trim: true,
    },
    password: {
      type: String,
      required: [true, "Password is required"],
      minlength: 6,
      select: false, // excluded from query results by default
    },
    avatar: {
      type: String,
      default: "",
    },
```

```
    role: {
      type: String,
      enum: ["user", "admin"],
      default: "user",
    },
  },
  { timestamps: true }
);

const User = mongoose.model("User", userSchema);

module.exports = User;
```

`unique: true` on `email` creates a unique index at the database level; it is not a validation rule by itself and needs `unique` indexes to actually be built (Mongoose does this automatically on model compilation in development).

`select: false` on `password` means `User.find()` never returns the password hash by default. You must explicitly opt in with `.select("+password")` when you need it, such as during login.

CHAPTER 14

Complete CRUD: The Task Resource End to End

This chapter wires together everything from Chapters 9-13 into a full, working CRUD feature for the Task resource — model, routes, controller, service, and the React pieces that consume the API.

14.1 Backend: Routes

```
// routes/task.routes.js
const express = require("express");
const {
  createTask,
  getTasks,
  getTaskById,
  updateTask,
  deleteTask,
} = require("../controllers/task.controller");
const { protect } = require("../middleware/auth.middleware");

const router = express.Router();

router.use(protect);

router.post("/", createTask);
router.get("/", getTasks);
router.get("/:id", getTaskById);
router.put("/:id", updateTask);
router.delete("/:id", deleteTask);

module.exports = router;
```

14.2 Backend: Service Layer

```
// services/task.service.js
const Task = require("../models/Task");

const createTask = async (data, userId) => {
  return Task.create({ ...data, user: userId });
};

const getTasks = async (userId, { page = 1, limit = 10, status }) => {
  const filter = { user: userId };
  if (status) filter.status = status;

  const tasks = await Task.find(filter)
    .sort({ createdAt: -1 })
    .skip((Number(page) - 1) * Number(limit))
    .limit(Number(limit));
```

```
  const total = await Task.countDocuments(filter);
  return { tasks, total, page: Number(page), limit: Number(limit) };
};

const getTaskById = async (id, userId) => {
  const task = await Task.findOne({ _id: id, user: userId });
  if (!task) {
    const err = new Error("Task not found");
    err.statusCode = 404;
    throw err;
  }
  return task;
};

const updateTask = async (id, userId, data) => {
  const task = await Task.findOneAndUpdate(
    { _id: id, user: userId },
    data,
    { new: true, runValidators: true }
  );
  if (!task) {
    const err = new Error("Task not found");
    err.statusCode = 404;
    throw err;
  }
  return task;
};

const deleteTask = async (id, userId) => {
  const task = await Task.findOneAndDelete({ _id: id, user: userId });
  if (!task) {
    const err = new Error("Task not found");
    err.statusCode = 404;
    throw err;
  }
  return task;
};

module.exports = { createTask, getTasks, getTaskById, updateTask, deleteTask };
```

14.3 Backend: Controller

```
// controllers/task.controller.js
const taskService = require("../services/task.service");

const createTask = async (req, res, next) => {
  try {
    const task = await taskService.createTask(req.body, req.user.id);
    res.status(201).json(task);
  } catch (err) {
    next(err);
  }
};

const getTasks = async (req, res, next) => {
  try {
    const result = await taskService.getTasks(req.user.id, req.query);
```

```
    res.status(200).json(result);
  } catch (err) {
    next(err);
  }
};

const getTaskById = async (req, res, next) => {
  try {
    const task = await taskService.getTaskById(req.params.id, req.user.id);
    res.status(200).json(task);
  } catch (err) {
    next(err);
  }
};

const updateTask = async (req, res, next) => {
  try {
    const task = await taskService.updateTask(req.params.id, req.user.id, req.body);
    res.status(200).json(task);
  } catch (err) {
    next(err);
  }
};

const deleteTask = async (req, res, next) => {
  try {
    await taskService.deleteTask(req.params.id, req.user.id);
    res.status(200).json({ message: "Task deleted" });
  } catch (err) {
    next(err);
  }
};

module.exports = { createTask, getTasks, getTaskById, updateTask, deleteTask };
```

14.4 Frontend: taskService.js

```
// src/services/taskService.js
import api from "./api"; // preconfigured axios instance with baseURL + credentials

export const fetchTasks = (params) => api.get("/tasks", { params });
export const fetchTaskById = (id) => api.get(`/tasks/${id}`);
export const createTask = (data) => api.post("/tasks", data);
export const updateTask = (id, data) => api.put(`/tasks/${id}`, data);
export const deleteTask = (id) => api.delete(`/tasks/${id}`);
```

14.5 Frontend: TaskForm.jsx

```
import { useState } from "react";
import { createTask, updateTask } from "../services/taskService";

const initialState = { title: "", description: "", status: "todo", priority: "medium"
    , dueDate: "" };
```

```
const TaskForm = ({ existingTask, onSaved }) => {
  const [form, setForm] = useState(existingTask || initialState);
  const [saving, setSaving] = useState(false);

  const handleChange = (e) => {
    setForm({ ...form, [e.target.name]: e.target.value });
  };

  const handleSubmit = async (e) => {
    e.preventDefault();
    setSaving(true);
    try {
      if (existingTask) {
        await updateTask(existingTask._id, form);
      } else {
        await createTask(form);
      }
      onSaved();
    } finally {
      setSaving(false);
    }
  };

  return (
    <form onSubmit={handleSubmit}>
      <input name="title" value={form.title} onChange={handleChange} placeholder="
          Title" required />
      <textarea name="description" value={form.description} onChange={handleChange}
          placeholder="Description" />
      <select name="status" value={form.status} onChange={handleChange}>
        <option value="todo">To Do</option>
        <option value="progress">In Progress</option>
        <option value="completed">Completed</option>
      </select>
      <select name="priority" value={form.priority} onChange={handleChange}>
        <option value="low">Low</option>
        <option value="medium">Medium</option>
        <option value="high">High</option>
      </select>
      <input type="date" name="dueDate" value={form.dueDate} onChange={handleChange}
          />
      <button type="submit" disabled={saving}>{saving ? "Saving..." : "Save Task"}</
          button>
    </form>
  );
};

export default TaskForm;
```

14.6 Frontend: TaskList.jsx

```
import { useEffect, useState } from "react";
import { fetchTasks, deleteTask } from "../services/taskService";
import TaskCard from "./TaskCard";

const TaskList = () => {
  const [tasks, setTasks] = useState([]);
```

```
  const [loading, setLoading] = useState(true);

  const loadTasks = async () => {
    setLoading(true);
    const { data } = await fetchTasks({ page: 1, limit: 10 });
    setTasks(data.tasks);
    setLoading(false);
  };

  useEffect(() => {
    loadTasks();
  }, []);

  const handleDelete = async (id) => {
    await deleteTask(id);
    setTasks((prev) => prev.filter((t) => t._id !== id));
  };

  if (loading) return <p>Loading tasks...</p>;

  return (
    <div className="task-list">
      {tasks.map((task) => (
        <TaskCard key={task._id} task={task} onDelete={() => handleDelete(task._id)}
            />
      ))}
    </div>
  );
};

export default TaskList;
```

14.7 End-to-End Flows

Create Task Flow

```
TaskForm submit
      |
      v
createTask(form)  ------> POST /api/tasks  (axios, withCredentials)
      |
      v
protect middleware -----> verifies auth cookie, sets req.user
      |
      v
task.controller.createTask -> reads req.body, calls service
      |
      v
task.service.createTask ---> Task.create({...data, user: userId})
      |
      v
MongoDB inserts document
      |
      v
```

```
201 response -----------> onSaved() -> TaskList reloads
```

Delete Task Flow

```
TaskCard "Delete" click
     |
     v
deleteTask(id)  --------> DELETE /api/tasks/:id
     |
     v
protect middleware -----> req.user set
     |
     v
task.controller.deleteTask -> calls service with id + userId
     |
     v
task.service.deleteTask --> Task.findOneAndDelete({_id, user})
     |
     v
200 response ------------> setTasks(prev => filter out id) (optimistic UI update)
```

Every service function scopes queries with `user: userId`. Without this, any authenticated user could read, update, or delete another user's tasks just by guessing an `_id`.

CHAPTER 15

Database Relationships

15.1 One-to-One: User and Profile

One-to-One

```
User Document                    Profile Document
--------------                   -----------------
_id: "u1"                        _id: "p1"
name: "Asha"                     user: "u1"  (ObjectId ref User)
email: "asha@mail.com"           bio: "Full-stack developer"
                                  location: "Kolkata"
```

```
// models/Profile.js
const profileSchema = new mongoose.Schema({
  user: { type: mongoose.Schema.Types.ObjectId, ref: "User", required: true, unique:
     true },
  bio: String,
  location: String,
});
```

15.2 One-to-Many: User and Tasks

One-to-Many

```
User "u1"
   |
   |-- Task "t1"  { user: "u1", title: "Fix login bug" }
   |-- Task "t2"  { user: "u1", title: "Write tests"    }
   `-- Task "t3"  { user: "u1", title: "Deploy to prod"  }
```

This is exactly the `Task.user` field from Chapter 13: each Task document stores a single `ObjectId` pointing back to its owning User.

15.3 populate(): Resolving References

By default, Mongoose stores and returns only the raw `ObjectId` for a `ref` field. `.populate()` replaces that id with the actual referenced document.

```
// Without populate
const task = await Task.findById(taskId);

// With populate - only pull name and email from the User document
const task = await Task.find().populate("user", "name email");
```

15.3.1 Before populate() — raw ObjectId

```
{
  "_id": "665f1a2b3c4d5e6f7a8b9c0d",
  "title": "Fix login bug",
  "status": "todo",
  "user": "664e0a1b2c3d4e5f6a7b8c9d"
}
```

15.3.2 After populate("user", "name email")

```
{
  "_id": "665f1a2b3c4d5e6f7a8b9c0d",
  "title": "Fix login bug",
  "status": "todo",
  "user": {
    "_id": "664e0a1b2c3d4e5f6a7b8c9d",
    "name": "Asha Verma",
    "email": "asha@mail.com"
  }
}
```

Passing `"name email"` as the second argument to `.populate()` is a projection — it limits which fields of the referenced document are returned, keeping the payload small and avoiding accidental exposure of fields like `password`.

15.4 Many-to-Many: Users and Projects

A many-to-many relationship needs an array of references on at least one side — here, a Project keeps an array of member User ids.

```
// models/Project.js
const projectSchema = new mongoose.Schema(
  {
    name: { type: String, required: true },
    members: [{ type: mongoose.Schema.Types.ObjectId, ref: "User" }],
  },
  { timestamps: true }
);
```

Many-to-Many

```
Project "Website Redesign"          Project "Mobile App"
   members: [u1, u2, u3]               members: [u2, u4]

User u1 ---belongs to---> Website Redesign
User u2 ---belongs to---> Website Redesign, Mobile App
User u3 ---belongs to---> Website Redesign
User u4 ---belongs to---> Mobile App
```

```
const project = await Project.findById(projectId).populate("members", "name email");
```

```
// project.members is now an array of user objects, not ids:
// members: [
// { _id: "u1", name: "Asha", email: "asha@mail.com" },
// { _id: "u2", name: "Ravi", email: "ravi@mail.com" },
// ...
// ]
```

`.populate()` works the same way whether the reference field holds a single `ObjectId` or an array of them — Mongoose detects the schema type automatically.

Populate runs an extra query behind the scenes. Populating deeply nested or large arrays on every request can hurt performance — only populate the fields the response actually needs.

CHAPTER 16

Complete Authentication System (Overview)

This chapter builds the shape of the authentication feature — register, login, logout, and "get current user" — at a flow level. Password hashing details (bcrypt) and token mechanics (JWT) are covered in depth in the following chapters; here we focus on how the pieces fit together.

16.1 Register Flow

REGISTER Flow

```
React (RegisterForm)
     |
     |  POST /api/auth/register  { name, email, password }
     v
Validate input              (required fields, email format, password length)
     |
     v
Hash password                (bcrypt.hash - see Chapter 17)
     |
     v
Save User document           (User.create with hashed password)
     |
     v
Generate auth token           (jwt.sign - see Chapter 18)
     |
     v
Response 201                  (user data + token/cookie set)
```

16.2 Login Flow

LOGIN Flow

```
React (LoginForm)
     |
     |  POST /api/auth/login  { email, password }
     v
Find user by email           (User.findOne({ email }).select("+password"))
     |
     v
Compare password              (bcrypt.compare - see Chapter 17)
     |
     v
Generate JWT                  (jwt.sign({ id: user._id }, SECRET))
     |
     v
```

```
  Set httpOnly cookie            (res.cookie("token", jwt, { httpOnly: true }))
       |
       v
  Authenticated user             (React stores user in context/state)
```

16.3 authController.js Skeleton

```
// controllers/auth.controller.js
const authService = require("../services/auth.service");

// POST /api/auth/register
const register = async (req, res, next) => {
  try {
    const { name, email, password } = req.body;
    // hash password – see Chapter 17
    // create user – see Chapter 13 (User model)
    // generate token – see Chapter 18
    const { user, token } = await authService.register({ name, email, password });

    res.cookie("token", token, { httpOnly: true });
    res.status(201).json({ user });
  } catch (err) {
    next(err);
  }
};

// POST /api/auth/login
const login = async (req, res, next) => {
  try {
    const { email, password } = req.body;
    // find user + compare password – see Chapter 17
    // generate JWT – see Chapter 18
    const { user, token } = await authService.login({ email, password });

    res.cookie("token", token, { httpOnly: true });
    res.status(200).json({ user });
  } catch (err) {
    next(err);
  }
};

// POST /api/auth/logout
const logout = async (req, res, next) => {
  try {
    // clear the auth cookie – see Chapter 19
    res.clearCookie("token");
    res.status(200).json({ message: "Logged out" });
  } catch (err) {
    next(err);
  }
};

// GET /api/auth/me
const getMe = async (req, res, next) => {
  try {
    // req.user is set by the protect middleware – see Chapter 19
```

```
    const user = await authService.getCurrentUser(req.user.id);
    res.status(200).json({ user });
  } catch (err) {
    next(err);
  }
};

module.exports = { register, login, logout, getMe };
```

16.4 Auth Routes

```
// routes/auth.routes.js
const express = require("express");
const { register, login, logout, getMe } = require("../controllers/auth.controller");
const { protect } = require("../middleware/auth.middleware");

const router = express.Router();

router.post("/register", register);
router.post("/login", login);
router.post("/logout", logout);
router.get("/me", protect, getMe);

module.exports = router;
```

16.5 What Comes Next

Topic	Covered in
Password hashing with bcrypt	Chapter 17
JWT generation and verification	Chapter 18
The `protect` middleware and httpOnly cookies	Chapter 19

Notice that `register` and `login` both end the same way: a signed token is issued and placed in an httpOnly cookie. This is the pattern the rest of the authentication chapters build on directly.

Never return the password field (hashed or not) in any auth response. The `User` schema's `select: false` on `password` (Chapter 13) protects against this by default, but always double-check the shape of `user` objects sent to the client.

CHAPTER 17

Password Hashing with bcrypt

A database full of user records is one breach away from being public. If passwords are stored as plain text, that breach instantly compromises every account – and because people reuse passwords, it compromises their accounts on other sites too. This chapter covers the one non-negotiable rule of authentication: passwords are never stored as-is.

Never store passwords in plain text, and never store them encrypted with a reversible cipher either. Encryption implies a key that can decrypt the value back to the original password – if that key leaks, every password leaks with it. Hashing is one-way: there is no key that turns a hash back into the original password. This is why bcrypt, not AES or a custom cipher, is the right tool for passwords.

17.1 Hashing and Salt

A hash function takes an input of any length and produces a fixed-length output. The same input always produces the same output, but there is no practical way to reverse it. If two users pick the password `password123`, a plain hash function (like SHA-256) would produce the *same* hash for both – which lets an attacker use precomputed lookup tables ("rainbow tables") to crack thousands of hashes at once.

A **salt** fixes this. It is random data mixed into the password before hashing, so the same password produces a different hash every time. bcrypt generates and stores this salt automatically as part of the hash string itself, so you never manage salts manually.

bcrypt Hashing Flow

```
Plain password: "MySecret123"
       |
       v
bcrypt generates a random salt
       |
       v
Salt + password run through the bcrypt algorithm
(repeated for 2^rounds iterations)
       |
       v
Resulting hash stored in DB:
$2b$10$N9qo8uLOickgx2ZMRZoMy...
(algorithm + rounds + salt + hash, all in one string)
```

17.2 Hashing on Registration

Install bcrypt and hash the password before saving the user document.

```
// controllers/authController.js
```

```
import bcrypt from "bcrypt";
import User from "../models/User.js";

export const registerUser = async (req, res, next) => {
  try {
    const { name, email, password } = req.body;

    const existing = await User.findOne({ email });
    if (existing) {
      return res.status(409).json({ success: false, message: "Email already in use" })
          ;
    }

    const salt = 10; // salt rounds
    const hashedPassword = await bcrypt.hash(password, salt);

    const user = await User.create({
      name,
      email,
      password: hashedPassword,
      role: "user",
    });

    res.status(201).json({
      success: true,
      data: { id: user._id, name: user.name, email: user.email, role: user.role },
    });
  } catch (err) {
    next(err);
  }
};
```

17.2.1 The Salt Rounds Tradeoff

The second argument to `bcrypt.hash` is the **cost factor** (commonly called salt rounds). It controls how many times the hashing algorithm loops internally – work scales as 2^{rounds}.

Rounds	Effect	Tradeoff
8–10	Fast (a few ms to 100ms per hash)	Weaker against brute force, fine for most apps
12	Slower (250ms per hash)	Better resistance, noticeable login latency
14+	Very slow (seconds)	Strong resistance, but hurts login UX and server throughput

10 is the widely used default and a reasonable choice for a typical Node.js API – it costs bcrypt roughly 50–100ms per hash on modern hardware, which is imperceptible to a user logging in but expensive enough to make brute forcing millions of guesses impractical.

17.3 Verifying on Login

You never “unhash” a password to compare it. Instead, bcrypt re-hashes the submitted password using the salt embedded in the stored hash, and compares the results.

```
// controllers/authController.js
export const loginUser = async (req, res, next) => {
  try {
    const { email, password } = req.body;

    const user = await User.findOne({ email });
    if (!user) {
      return res.status(401).json({ success: false, message: "Invalid credentials" });
    }

    const match = await bcrypt.compare(password, user.password);
    if (!match) {
      return res.status(401).json({ success: false, message: "Invalid credentials" });
    }

    // generateToken + res.cookie happen here (see Chapters 18–19)
    res.status(200).json({ success: true, data: { id: user._id, email: user.email }
        });
  } catch (err) {
    next(err);
  }
};
```

Return the same generic “Invalid credentials” message whether the email does not exist or the password is wrong. Distinguishing the two tells an attacker which emails are registered in your system.

17.4 Alternative: Hashing in a Mongoose Pre-Save Hook

Instead of hashing manually in every controller that creates or updates a user, you can centralize it in the User schema using a `pre("save")` hook. This guarantees hashing happens no matter where a user document is saved from.

```
// models/User.js
import mongoose from "mongoose";
import bcrypt from "bcrypt";

const userSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  password: { type: String, required: true },
  avatar: { type: String, default: "" },
  role: { type: String, enum: ["user", "manager", "admin"], default: "user" },
});

userSchema.pre("save", async function (next) {
  if (!this.isModified("password")) return next(); // skip if unchanged

  const salt = await bcrypt.genSalt(10);
  this.password = await bcrypt.hash(this.password, salt);
  next();
});

userSchema.methods.comparePassword = function (candidate) {
  return bcrypt.compare(candidate, this.password);
```

```
};

export default mongoose.model("User", userSchema);
```

17.4.1 Why `isModified("password")` Matters

Without this check, every time a user document is saved – even to update an unrelated field like `avatar` – the hook would re-hash the *already-hashed* password. That produces a hash-of-a-hash, permanently locking the user out because future logins compare against the wrong value. `isModified("password")` ensures the hook only re-hashes when the raw password field was actually changed on this save.

With the pre-save hook approach, the controller simplifies to `User.create({ name, email, password, role })` – no manual `bcrypt.hash` call needed. Login still uses `user.comparePassword(password)` (or `bcrypt.compare` directly) exactly as shown above.

WHAT: bcrypt hashes passwords with a per-password random salt before storage, and compares candidate passwords against the stored hash on login.

WHY: Plain-text or reversibly-encrypted passwords turn any database leak into a full account compromise; one-way hashing with salt makes stored values useless to an attacker even if stolen.

HOW: Hash with `bcrypt.hash(password, 10)` on registration (or via a `pre("save")` hook guarded by `isModified("password")`), and verify with `bcrypt.compare(password, user.password)` on login.

CHAPTER 18

JWT Authentication

Once a user's password is verified, the server needs a way to remember that this browser is "logged in" on every subsequent request – without querying the database on every single request just to check a session table. A **JSON Web Token (JWT)** solves this by packing identity information into a signed, self-contained string that the server can verify without storing anything.

18.1 Anatomy of a JWT

A JWT is three base64url-encoded segments separated by dots: `header.payload.signature`.

Segment	Contents
Header	Metadata: the signing algorithm (e.g. `HS256`) and token type.
Payload	Claims – the actual data, e.g. `{ id: userId, iat, exp }`. Readable by anyone, not encrypted.
Signature	`HMACSHA256(header + payload, JWT_SECRET)`. Proves the token was issued by your server and hasn't been tampered with.

The payload is base64-encoded, not encrypted – anyone with the token can decode and read it (try pasting one into jwt.io). Never put passwords, secrets, or sensitive personal data in a JWT payload. Only put identifiers like a user ID.

18.1.1 Expiration and Signing

Every token should carry an `exp` (expiration) claim so a stolen or leaked token becomes useless after a fixed window. The signature is produced using a secret key – `process.env.JWT_SECRET` – that only your server knows. If that secret leaks, anyone can forge valid tokens for any user, so treat it like a password and never commit it to source control.

18.2 The Authentication Flow

JWT Authentication Flow

```
1. Browser -> POST /api/auth/login (email, password)
2. Express -> Credentials verified (bcrypt.compare)
3. Express -> JWT generated (jwt.sign, signed with JWT_SECRET)
4. Express -> JWT sent to browser (in an HttpOnly cookie)
5. Browser -> stores cookie automatically, sends it on every request
6. Browser -> GET /api/tasks  (cookie attached automatically)
7. Middleware -> reads token, verifies with jwt.verify
8. Middleware -> decoded payload -> req.user = user document
9. Protected Controller -> runs, trusting req.user
```

18.3 Generating a Token

```
// utils/generateToken.js
import jwt from "jsonwebtoken";

export const generateToken = (userId) => {
  return jwt.sign(
    { id: userId },
    process.env.JWT_SECRET,
    { expiresIn: "7d" }
  );
};
```

Used right after credentials are verified in the login controller:

```
// controllers/authController.js
import { generateToken } from "../utils/generateToken.js";

export const loginUser = async (req, res, next) => {
  try {
    const { email, password } = req.body;
    const user = await User.findOne({ email });
    if (!user || !(await bcrypt.compare(password, user.password))) {
      return res.status(401).json({ success: false, message: "Invalid credentials" });
    }

    const token = generateToken(user._id);

    res.cookie("token", token, {
      httpOnly: true,
      maxAge: 7 * 24 * 60 * 60 * 1000,
    });

    res.status(200).json({
      success: true,
      data: { id: user._id, name: user.name, role: user.role },
    });
  } catch (err) {
    next(err);
  }
};
```

Full cookie configuration (`secure`, `sameSite`, and why `httpOnly` matters) is covered in the next chapter. For now, focus on how the token itself is created and verified.

18.4 Verifying a Token

```
// middleware/authMiddleware.js
import jwt from "jsonwebtoken";

export const protect = async (req, res, next) => {
  const token = req.cookies.token;
  if (!token) {
    return res.status(401).json({ success: false, message: "Not authenticated" });
```

```
  }

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.userId = decoded.id; // full lookup covered in Chapter 20
    next();
  } catch (err) {
    return res.status(401).json({ success: false, message: "Invalid or expired token"
        });
  }
};
```

`jwt.verify` checks the signature against `JWT_SECRET` and automatically rejects the token if the `exp` claim has passed – either case throws, which is why it must be wrapped in `try/catch`.

Larger production systems often split this into two tokens: a short-lived **access token** (minutes) used on every request, and a long-lived **refresh token** (days/weeks) used only to silently obtain a new access token when the old one expires. This limits the damage window if an access token leaks, at the cost of extra complexity (a refresh endpoint, refresh token storage/rotation, revocation logic). This book uses a single 7-day token throughout to keep the implementation focused – but know that the two-token pattern is the natural next step for a larger production system.

WHAT: A JWT is a signed token containing a user identifier and expiration, generated on login and verified on every protected request.

WHY: It lets the server confirm "who is this request from" without a database lookup on every request or server-side session storage, while still expiring automatically.

HOW: Sign with `jwt.sign({id}, JWT_SECRET, {expiresIn})` at login, store it in a cookie, and verify with `jwt.verify(token, JWT_SECRET)` inside middleware before any protected controller runs.

CHAPTER 19

Secure Authentication Cookies

Generating a JWT is only half the job – how it travels between server and browser matters just as much. A token sent carelessly (e.g. stored in `localStorage` and attached manually via JavaScript) is exposed to any script running on the page, including malicious ones injected through an XSS vulnerability. This chapter covers storing the JWT in a properly configured cookie so the browser handles it safely and automatically.

19.1 Setting the Cookie

First, parse incoming cookies with the `cookie-parser` middleware:

```
// app.js
import cookieParser from "cookie-parser";
app.use(cookieParser());
```

Then set the cookie when issuing a token at login:

```
res.cookie("token", token, {
  httpOnly: true,
  secure: process.env.NODE_ENV === "production",
  sameSite: "lax",
  maxAge: 7 * 24 * 60 * 60 * 1000, // 7 days, in milliseconds
});
```

And clear it on logout:

```
// controllers/authController.js
export const logoutUser = (req, res) => {
  res.clearCookie("token");
  res.status(200).json({ success: true, message: "Logged out" });
};
```

`clearCookie` must be called with the same options (path, domain) that were used when the cookie was set, or the browser will not recognize it as the same cookie and will fail to remove it.

19.2 Why Each Cookie Option Exists

Option	What it does	Why it matters
`httpOnly`	Blocks `document.cookie` from reading this cookie in JavaScript.	Prevents an XSS attack (malicious injected script) from stealing the token.
`secure`	Cookie is only sent over HTTPS connections.	Prevents the token from being visible to anyone snooping on plain HTTP traffic.
`sameSite`	Controls whether the cookie is sent on cross-site requests. `"lax"` sends it on top-level navigation but not on cross-site subrequests/forms.	Mitigates CSRF by stopping other sites from silently riding on the user's session.
`maxAge`	How long the cookie persists before the browser deletes it.	Matches the JWT's own `expiresIn` so an expired token isn't kept around uselessly.

`secure` is conditioned on `NODE_ENV === "production"` because local development typically runs over plain `http://localhost`. Setting `secure: true` unconditionally would silently stop the cookie from being set at all during development, since browsers refuse to send secure cookies over non-HTTPS origins.

19.3 The Frontend Side: withCredentials

Cookies are domain-scoped and, for cross-origin requests (React dev server on port 5173 talking to an API on port 5000, or a separate frontend/backend domain in production), the browser will not attach or store cookies unless explicitly told to. Configure the shared axios instance once:

```
// src/api/axios.js
import axios from "axios";

const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL,
  withCredentials: true, // send and accept cookies cross-origin
});

export default api;
```

`withCredentials: true` on the frontend only works if the backend's CORS configuration explicitly sets `credentials: true` and lists the exact frontend origin (never a wildcard `*`) in `origin`. The two settings must match on both sides, or the browser silently blocks the cookie. Full CORS setup is covered in Chapter 26.

WHAT: Storing the JWT in a cookie configured with `httpOnly`, `secure`, and `sameSite`, rather than in localStorage or a plain cookie.

WHY: These three flags close off the two most common ways tokens get stolen or misused: XSS reading it from JavaScript, and network sniffing or CSRF riding along with it.

HOW: Set it with `res.cookie("token", token, {httpOnly, secure, sameSite, maxAge})` on login, clear it with `res.clearCookie("token")` on logout, and pair it with `withCredentials: true` on the frontend plus a matching CORS config on the backend.

CHAPTER 20

Auth Middleware for Protected APIs

Chapters 18 and 19 covered issuing and storing a JWT. This chapter puts it to work: a reusable Express middleware that stands in front of any route that should only be reachable by a logged-in user, verifies the token, and loads the actual user document so controllers can trust `req.user`.

Protected Request Flow

```
Request (with "token" cookie)
        |
        v
Auth Middleware (protect)
        |
        v
JWT Verification (jwt.verify against JWT_SECRET)
        |
        v
User Lookup (User.findById(decoded.id))
        |
        v
req.user = user document (password excluded)
        |
        v
next() -> Controller (trusts req.user, no re-checking)
```

20.1 The `protect` Middleware

```
// middleware/authMiddleware.js
import jwt from "jsonwebtoken";
import User from "../models/User.js";

export const protect = async (req, res, next) => {
  const token = req.cookies.token;

  if (!token) {
    return res.status(401).json({ success: false, message: "Not authenticated" });
  }

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);

    const user = await User.findById(decoded.id).select("-password");
    if (!user) {
      return res.status(401).json({ success: false, message: "User no longer exists"
          });
    }

    req.user = user;
```

```
    next();
  } catch (err) {
    return res.status(401).json({ success: false, message: "Invalid or expired token"
        });
  }
};
```

20.1.1 Walking Through Each Failure Case

Scenario	Result
No cookie sent at all	`req.cookies.token` is `undefined` – immediate 401, never touches `jwt.verify`.
Token tampered with or signed by a different secret	`jwt.verify` throws a `JsonWebTokenError` – caught, 401.
Token past its `exp` claim	`jwt.verify` throws a `TokenExpiredError` – caught, 401.
Token valid, but user was deleted after it was issued	`jwt.verify` succeeds, `User.findById` returns `null` – explicit 401.
Token valid, user exists	`req.user` is set, `next()` passes control to the controller.

`.select("-password")` excludes the hashed password from the document attached to `req.user`. Even though it's a hash and not the plain password, there's no reason for it to ever leave the database layer.

20.2 Using the Middleware

Apply `protect` to any route that requires a logged-in user, simply by inserting it before the controller in the route definition:

```
// routes/taskRoutes.js
import express from "express";
import { protect } from "../middleware/authMiddleware.js";
import { getTasks, createTask } from "../controllers/taskController.js";

const router = express.Router();

router.get("/tasks", protect, getTasks);
router.post("/tasks", protect, createTask);

export default router;
```

Inside `getTasks`, the controller can now safely reference `req.user` without re-verifying anything:

```
export const getTasks = async (req, res, next) => {
  try {
    const tasks = await Task.find({ user: req.user._id });
    res.status(200).json({ success: true, data: tasks });
  } catch (err) {
    next(err);
  }
```

```
};
```

Express runs middleware in the order listed. Because `protect` appears before `getTasks` in the route definition, it always executes first – if it doesn't call `next()`, the controller never runs. This is what makes the route "protected."

WHAT: An Express middleware that reads the JWT cookie, verifies it, loads the corresponding user, and attaches it to `req.user` before letting the request continue.

WHY: Controllers need a trustworthy, verified identity without each one re-implementing token verification and database lookups themselves.

HOW: Insert `protect` as a route parameter before the controller function: `router.get("/tasks", protect, getTasks);` any missing, invalid, or expired token short-circuits with a 401 before the controller ever runs.

CHAPTER 21

Authorization: Roles and Permissions

Knowing *who* is making a request is only half the security story. Knowing *what that person is allowed to do* is the other half. A logged in user and a logged-in admin both pass the `protect` middleware from Chapter 20 – but only one of them should be able to delete someone else's task. That distinction is authorization.

21.1 Authentication vs Authorization

These two terms get used interchangeably in casual conversation, but they answer different questions and are implemented as separate middleware layers.

	Authentication	**Authorization**
Question answered	"Who are you?"	"What are you allowed to do?"
When it runs	Every protected request, first	After authentication, before the controller
Implemented by	`protect` middleware (Ch. 18/20)	`authorize(...roles)` middleware (this chapter)
Failure response	401 Unauthorized	403 Forbidden
Based on	Valid JWT / session	`req.user.role` (or other permission data)

A precise mental shortcut: **401** means "I don't know who you are" (or your proof of identity is invalid). **403** means "I know exactly who you are, and the answer is still no."

21.2 The Role Field

Recall the User schema from Chapter 13 – the `role` field constrains every user to one of three values:

```
// models/User.js (relevant field)
role: { type: String, enum: ["user", "manager", "admin"], default: "user" },
```

- **user** – can manage only their own tasks.
- **manager** – can view and update tasks across their team.
- **admin** – full access, including deleting any task or user.

21.3 The `authorize` Middleware Factory

Rather than writing a separate middleware for every role combination, write one factory function that takes the allowed roles as arguments and returns a middleware tailored to that route.

```
// middleware/authorize.js
export const authorize = (...allowedRoles) => {
  return (req, res, next) => {
    if (!req.user) {
      return res.status(401).json({ success: false, message: "Not authenticated" });
    }

    if (!allowedRoles.includes(req.user.role)) {
      return res.status(403).json({
        success: false,
        message: `Role '${req.user.role}' is not permitted to perform this action`,
      });
    }

    next();
  };
};
```

Because `authorize` always runs after `protect`, it can rely on `req.user` already being set – the `!req.user` check is just a defensive guard in case the middleware order is ever misconfigured.

21.4 Using It on Routes

```
// routes/taskRoutes.js
import { protect } from "../middleware/authMiddleware.js";
import { authorize } from "../middleware/authorize.js";
import { deleteTask, getTasks } from "../controllers/taskController.js";

router.get("/tasks", protect, getTasks);

router.delete("/tasks/:id", protect, authorize("admin", "manager"), deleteTask);
```

Reading this route top to bottom: a request must first pass `protect` (prove identity), then `authorize("admin", "manager")` (prove permission), before `deleteTask` ever executes. A plain `user` role gets a 403 and the delete never touches the database.

Because `authorize` takes roles as arguments, the same middleware function covers every permission combination in the app – `authorize("admin")` for admin-only routes, `authorize("admin", "manager")` for shared access, with no duplicated logic.

Role checks alone don't cover ownership. A `manager` passing `authorize("manager")` still shouldn't necessarily be able to delete a task belonging to a different team. For ownership-level rules, add an explicit check inside the controller (e.g. comparing `task.user` against `req.user._id`) in addition to the role middleware.

WHAT: An `authorize(...roles)` middleware factory that checks `req.user.role` against a list of permitted roles for a given route.

WHY: Authentication only proves identity; without a separate authorization check, any logged-in user – regardless of role – could call sensitive endpoints like deleting tasks or managing other users.

HOW: Chain it after `protect` in the route definition: `router.delete("/tasks/:id", protect, authorize("admin", "manager"), deleteTask)`, returning 403 for any role not in the allowed list.

CHAPTER 22

Google OAuth Login

Email/password login works, but many users expect a ”Sign in with Google” button too – one less password to create and remember. This chapter walks through wiring Google OAuth into the Task Management System so that, after Google verifies who the user is, your own backend still issues the same JWT cookie used everywhere else in the app. Nothing downstream (the `protect` middleware, `authorize`, controllers) needs to know or care whether a user logged in with a password or with Google.

22.1 The Big Picture

The key idea: your React app never handles the user's Google password, and your Express server never talks to Google's login page directly. Instead, Google's own SDK on the frontend handles the login UI and hands your backend a signed **ID token** proving the user's identity, which your backend verifies with Google before trusting it.

Google OAuth Flow

```
1. React renders Google's "Sign in with Google" button
       |  user clicks, Google popup/redirect handles login
       v
2. Google -> Authorization (user approves access)
       |
       v
3. Callback -> React receives an ID token from Google
       |
       v
4. React -> POST /api/auth/google  { idToken }
       |
       v
5. Express -> verifies token with Google
            (OAuth2Client.verifyIdToken)
       |
       v
6. Express -> Find user by email, or Create if first login
       |
       v
7. Express -> Generate JWT + set cookie (same as normal login)
       |
       v
8. React -> receives success response, user is authenticated
```

22.2 Google Cloud Console Setup

Before any code, register the app in the Google Cloud Console (APIs & Services → Credentials → Create OAuth Client ID):

- Choose **Web application** as the application type.
- Add your frontend origin (e.g. `http://localhost:5173`) under **Authorized JavaScript origins**.
- Add your callback/redirect URI under **Authorized redirect URIs**, if using the redirect-based flow rather than the popup-based Google Identity Services button.
- Copy the generated **Client ID** and **Client Secret**.

```
# .env
GOOGLE_CLIENT_ID=1234567890-abc123.apps.googleusercontent.com
GOOGLE_CLIENT_SECRET=GOCSPX-xxxxxxxxxxxxxxxxxxxxxxx
```

This is the single most common Google OAuth setup error. Google requires the redirect URI your app actually sends to *exactly* match one of the URIs registered in the Cloud Console – byte for byte. Common mismatches:

- `http://localhost:5173` registered, but the app runs on `http://localhost:5173/` (trailing slash) or a different port.
- `http` registered in development, but `https` used in production (these are two separate URIs to Google, not the same one).
- A subdomain difference, e.g. `app.example.com` registered but `www.app.example.com` used.

Fix it by copying the exact URI your app sends (visible in the error message or network tab) and adding that precise string to Authorized redirect URIs in the Cloud Console, or by correcting your app's configured redirect URI to match what's registered.

22.3 Backend: Verifying the ID Token

Using the `google-auth-library` package, the backend receives the ID token from the frontend and verifies it directly with Google's servers – this confirms the token is genuine and was issued for *your* Client ID.

```
// controllers/authController.js
import { OAuth2Client } from "google-auth-library";
import User from "../models/User.js";
import { generateToken } from "../utils/generateToken.js";

const client = new OAuth2Client(process.env.GOOGLE_CLIENT_ID);

export const googleLogin = async (req, res, next) => {
  try {
    const { idToken } = req.body;

    const ticket = await client.verifyIdToken({
      idToken,
      audience: process.env.GOOGLE_CLIENT_ID,
    });

    const payload = ticket.getPayload();
    const { email, name, picture } = payload;

    let user = await User.findOne({ email });
```

```
    if (!user) {
      // First-time Google login: create the account.
      // No password is set for OAuth-only accounts.
      user = await User.create({
        name,
        email,
        avatar: picture,
        password: undefined,
        role: "user",
      });
    }

    const token = generateToken(user._id);

    res.cookie("token", token, {
      httpOnly: true,
      secure: process.env.NODE_ENV === "production",
      sameSite: "lax",
      maxAge: 7 * 24 * 60 * 60 * 1000,
    });

    res.status(200).json({
      success: true,
      data: { id: user._id, name: user.name, email: user.email, role: user.role },
    });
  } catch (err) {
    next(err);
  }
};
```

```
// routes/authRoutes.js
router.post("/auth/google", googleLogin);
```

22.4 Handling the Password Field

Because Google-authenticated users never set a password on your system, the User schema's `password` field must be optional for this to work:

```
// models/User.js
password: { type: String, required: false },
```

If a user later tries to log in with email/password using an account that was created via Google (and therefore has no password hash), the login controller's `bcrypt.compare` call needs a guard: check `if (!user.password)` first and return a clear "This account uses Google Sign-In" message instead of attempting to compare against `undefined`.

22.5 Subsequent Logins

On every later login, the same `googleLogin` controller runs: `User.findOne({ email })` finds the existing account (skipping the creation branch), and a fresh JWT cookie is issued exactly as it was

the first time. From the rest of the application's perspective – middleware, controllers, the frontend's authenticated state – a Google-authenticated session and a password-authenticated session are indistinguishable; both end with the same signed JWT cookie.

Keep the Google verification logic isolated to this one controller. Every other part of the backend (`protect`, `authorize`, task controllers) only ever deals with your own JWT, never with Google tokens directly. This is what makes adding a second OAuth provider later (GitHub, Facebook) a matter of adding another controller, not rewriting existing auth logic.

WHAT: An endpoint that accepts a Google-issued ID token from the frontend, verifies it server-side with Google, finds or creates a matching user, and issues your app's own JWT cookie.

WHY: Letting the frontend alone decide "this user is authenticated with Google" would be trivially spoofable; server-side verification against Google's servers is the only trustworthy point, and re-using your existing JWT flow keeps the rest of the app provider-agnostic.

HOW: Verify with `OAuth2Client.verifyIdToken`, `findOrCreate` the user by email, then call the exact same `generateToken` + `res.cookie` steps used for normal login.

CHAPTER 23

Backend Validation

A form in React can disable the submit button until all fields look right. That's good UX – and it stops there. It is not security, and it is not data integrity. Anyone can open devtools, call the API directly with Postman, or write a script that hits your endpoints with whatever payload they want, completely bypassing every input you rendered in JSX.

Frontend validation is a convenience for honest users, not a trust boundary. The only code that runs on infrastructure you control is your backend – that is the only place a validation rule is actually enforced. Every endpoint must independently validate its input as if the frontend did not exist.

23.1 Frontend vs Backend Validation

	Frontend Validation	**Backend Validation**
Purpose	Fast feedback, better UX	Enforce data integrity and security
Runs where	Browser (React state, HTML attributes)	Server (Express middleware)
Can be bypassed?	Yes – devtools, Postman, curl, disabling JS	No – it's the last line before the database
Required?	Nice to have	Mandatory

23.2 Validating with express-validator

`express-validator` lets you declare validation rules as middleware, directly on the route, right next to the endpoint they protect.

```
// validators/taskValidator.js
import { body, validationResult } from "express-validator";

export const validateTask = [
  body("title")
    .trim()
    .notEmpty().withMessage("Title is required")
    .isString().withMessage("Title must be a string")
    .isLength({ min: 3 }).withMessage("Title must be at least 3 characters"),

  body("status")
    .optional()
    .isIn(["pending", "in-progress", "completed"])
    .withMessage("Status must be one of: pending, in-progress, completed"),

  (req, res, next) => {
    const errors = validationResult(req);
    if (!errors.isEmpty()) {
      return res.status(422).json({
```

```
      success: false,
      errors: errors.array().map((e) => ({ field: e.path, message: e.msg })),
    });
  }
  next();
 },
];
```

`validateTask` is an array of middleware: each `body(...)` chain runs first and attaches any errors to the request without throwing, then the final function checks `validationResult(req)` and either short-circuits with a 422 or calls `next()` to let the controller run.

422 (Unprocessable Entity) is the conventional status code for "the request was well-formed but failed validation rules," distinct from 400 (malformed request) or 401/403 (auth failures). Using it consistently makes frontend error handling predictable.

23.3 Wiring It Into the Route

Because `validateTask` is itself an array of middleware, Express happily spreads it into the route alongside `protect`:

```
// routes/taskRoutes.js
import { protect } from "../middleware/authMiddleware.js";
import { validateTask } from "../validators/taskValidator.js";
import { createTask } from "../controllers/taskController.js";

router.post("/tasks", protect, validateTask, createTask);
```

The request now passes through, in order: authentication (`protect`), then validation (`validateTask`), and only then does `createTask` run – by that point the controller can trust that `req.body.title` exists and `req.body.status` (if present) is one of the allowed enum values.

```
// controllers/taskController.js
export const createTask = async (req, res, next) => {
 try {
  const { title, status } = req.body; // already validated
  const task = await Task.create({ title, status, user: req.user._id });
  res.status(201).json({ success: true, data: task });
 } catch (err) {
  next(err);
 }
};
```

Keep validation rules close to the shape they're checking, but separate from business logic in the controller. If the Task model's schema already defines `enum: ["pending", "in-progress", "completed"]`, the validator and the schema are stating the same rule in two places – that duplication is acceptable here, because the validator's job is to reject bad input *before* a database round trip, with field-level error messages, rather than relying solely on a Mongoose `ValidationError` after the fact.

WHAT: Middleware-based request validation (using express-validator) that checks incoming data shape, types, and allowed values before a controller touches it.

WHY: Frontend checks can always be bypassed by anyone calling the API directly, so the backend is the only place invalid or malicious data can reliably be rejected before it reaches the database.

HOW: Chain validators like `body("title").notEmpty()` into an array with a final `validationResult` check, and insert that array into the route: `router.post("/tasks", protect, validateTask, createTask)`, returning 422 with field-level errors on failure.

CHAPTER 24

Global Error Handling

Scattered `try/catch` blocks that each format their own error response lead to inconsistent API output – one endpoint returns `{ error: "..." }`, another returns `{ message: "..." }`, and a third crashes the process entirely on an unhandled rejection. A single global error handling middleware fixes all three problems at once: every error, from anywhere in the app, funnels through one place that shapes the response consistently.

Error Handling Flow

```
Controller
   |  something goes wrong
   v
throw new AppError("Task not found", 404)
   |
   v
catch block -> next(error)
   |
   v
(skips all remaining normal middleware/routes)
   |
   v
Global Error Middleware  (app.use((err, req, res, next) => ...))
   |
   v
JSON Response  { success: false, message }
```

24.1 A Custom AppError Class

Plain `Error` objects don't carry an HTTP status code. A small subclass fixes that, letting any part of the app throw an error that already knows how it should be reported to the client.

```
// utils/AppError.js
class AppError extends Error {
  constructor(message, statusCode) {
    super(message);
    this.statusCode = statusCode;
    this.isOperational = true; // distinguishes expected errors from bugs
  }
}

export default AppError;
```

```
// controllers/taskController.js
import AppError from "../utils/AppError.js";

export const getTaskById = async (req, res, next) => {
  try {
```

```
    const task = await Task.findById(req.params.id);
    if (!task) {
      throw new AppError("Task not found", 404);
    }
    res.status(200).json({ success: true, data: task });
  } catch (err) {
    next(err);
  }
};
```

24.2 Avoiding Repetitive try/catch: asyncHandler

Writing `try { ... } catch (err) { next(err) }` in every single controller is repetitive and easy to forget. A small wrapper function eliminates it by catching any rejected promise automatically.

```
// utils/asyncHandler.js
const asyncHandler = (fn) => (req, res, next) => {
  Promise.resolve(fn(req, res, next)).catch(next);
};

export default asyncHandler;
```

```
// controllers/taskController.js
import asyncHandler from "../utils/asyncHandler.js";
import AppError from "../utils/AppError.js";

export const getTaskById = asyncHandler(async (req, res) => {
  const task = await Task.findById(req.params.id);
  if (!task) {
    throw new AppError("Task not found", 404);
  }
  res.status(200).json({ success: true, data: task });
});
```

Any thrown error, or any rejected promise from an `await` call inside the wrapped function, is automatically passed to `next(err)` – the controller body no longer needs its own `try/catch` at all.

24.3 The Global Error Middleware

Express recognizes a middleware with four parameters `(err, req, res, next)` as an error handler. It must be registered *after* all routes, so anything that calls `next(err)` anywhere in the app eventually reaches it.

```
// middleware/errorHandler.js
export const errorHandler = (err, req, res, next) => {
  let statusCode = err.statusCode || 500;
  let message = err.message || "Internal Server Error";

  // Mongoose: invalid ObjectId format, e.g. Task.findById("not-an-id")
  if (err.name === "CastError") {
    statusCode = 400;
    message = `Invalid ${err.path}: ${err.value}`;
  }

  // Mongoose: schema validation failed (required field, enum, etc.)
```

```
  if (err.name === "ValidationError") {
    statusCode = 400;
    message = Object.values(err.errors).map((e) => e.message).join(", ");
  }

  // MongoDB: duplicate key on a unique field, e.g. email already exists
  if (err.code === 11000) {
    statusCode = 409;
    const field = Object.keys(err.keyValue)[0];
    message = `${field} already exists`;
  }

  res.status(statusCode).json({
    success: false,
    message,
  });
};
```

```
// app.js
import { errorHandler } from "./middleware/errorHandler.js";

app.use("/api/auth", authRoutes);
app.use("/api/tasks", taskRoutes);

// must be registered after all routes
app.use(errorHandler);
```

Middleware order matters. If `errorHandler` is registered before the routes, Express never reaches it when a route calls `next(err)` – the request hangs or Express falls back to its own default (and much less helpful) error page.

24.4 Mapping Mongoose-Specific Errors

Error	Cause	Mapped status
`CastError`	Malformed value for a typed field, most often an invalid `ObjectId` in a URL param.	400
`ValidationError`	A document failed schema rules (`required`, `enum`, `minlength`, etc.) on save/create.	400
Code `11000`	Duplicate key write against a field with a `unique` index (e.g. registering an email that already exists).	409

Because these checks live inside the single `errorHandler`, no individual controller needs to know about Mongoose error internals – `Task.findById(badId)` throwing a `CastError` is turned into a clean `400 { message: "Invalid _id: ..." }` response automatically, regardless of which controller triggered it.

WHAT: A single error-handling middleware, registered last in app.js, that every thrown or forwarded error eventually reaches and formats into a consistent JSON response.

WHY: Without it, error handling is duplicated and inconsistent across every controller, and

raw Mongoose/MongoDB errors leak internal details or default to an unhelpful 500 instead of a meaningful status code.

HOW: Throw an `AppError(message, statusCode)` (or let asyncHandler catch any rejected promise), forward it with `next(err)`, and let `app.use((err, req, res, next) => ...)` map it – including special cases like `CastError`, `ValidationError`, and duplicate key code 11000 – to the right status and message.

CHAPTER 25

HTTP Status Codes Reference

25.1 Why Status Codes Matter

The HTTP status code is the first thing the frontend looks at before it even touches the response body. A consistent status-code contract lets the React layer make one decision early — "was this a success, a client mistake, or a server failure?" — and route the response accordingly: show data, show a field-level validation message, redirect to login, or show a generic error toast. When an API returns `200 OK` for everything (including errors buried in the JSON body), every component that calls it has to re-implement its own ad-hoc error detection. Picking the right code per situation, and using it consistently across every controller, is what makes `axios` interceptors and global error handlers (Chapter 17) possible in the first place.

25.2 Reference Table

Code	Meaning	Typical MERN Usage
200	OK	Successful `GET` request; data returned in the body.
201	Created	Successful `POST` request; a new resource (task, user) was created.
400	Bad Request	Malformed request — missing required field, invalid ID format, bad JSON.
401	Unauthorized	Missing, expired, or invalid JWT/session; user is not logged in.
403	Forbidden	User is authenticated but lacks permission (e.g. not the task owner, not an admin).
404	Not Found	Requested resource (task, user, route) does not exist.
409	Conflict	Duplicate data — e.g. registering with an email that already exists.
422	Unprocessable Entity	Request is well-formed but fails validation rules (bad email format, password too short).
500	Internal Server Error	Unexpected backend failure — DB connection lost, unhandled exception.

Many teams use `400` for *all* validation failures instead of `422`, and that is a perfectly valid convention too. What matters is picking one rule and applying it everywhere — the Task Management System in this book uses `400` for malformed requests and `422` specifically for schema/validation errors raised by `express-validator` or Mongoose validators.

Never return `200 OK` with an `{ "error": "..." }` body for a failed operation. It silently breaks any frontend code that checks `response.ok` or relies on axios's automatic rejection of non-2xx responses.

CHAPTER 26

CORS --- Connecting Different Origins

26.1 Why the Browser Blocks the Request

During local development the React app runs on `http://localhost:5173` (Vite's default port) and the Express API runs on `http://localhost:5000`. Even though both are "localhost", they are two different **origins** — an origin is the triple `scheme + host + port`, and any difference in any one of those three makes it a cross-origin request. Browsers enforce the **Same-Origin Policy**: JavaScript running on one origin cannot read responses from a different origin unless that origin explicitly says it is allowed to. This is a browser-side security rule, not a server-side one — tools like Postman or `curl` never hit this wall because they aren't bound by same-origin restrictions.

CORS (Cross-Origin Resource Sharing) is the mechanism the server uses to opt certain origins back in, by sending back headers like `Access-Control-Allow-Origin`.

26.1.1 Preflight Requests

For "non-simple" requests — anything using `PUT`, `PATCH`, `DELETE`, a JSON body, or custom headers like `Authorization` — the browser first sends an automatic `OPTIONS` request called a **preflight**. It asks the server "if I were to send this real request, would you allow it?" before sending the actual request. The server must respond to `OPTIONS` with the appropriate `Access-Control-Allow-*` headers, or the browser never sends the real request at all.

Preflight Flow

```
Browser (localhost:5173)              Server (localhost:5000)
        |                                      |
        |--- OPTIONS /api/tasks -------------->|  (preflight)
        |    Origin: localhost:5173            |
        |    Access-Control-Request-Method: PUT|
        |                                      |
        |<-- Access-Control-Allow-Origin ------|
        |<-- Access-Control-Allow-Methods -----|
        |<-- Access-Control-Allow-Credentials -|
        |                                      |
        |--- PUT /api/tasks/123 -------------->|  (actual request,
        |                                      |   only sent if
        |<-- 200 OK ---------------------------|   preflight passed)
```

26.2 Configuring CORS in Express

```
const cors = require("cors");

app.use(
  cors({
    origin: "http://localhost:5173", // exact frontend origin, not "*"
    credentials: true, // allow cookies / Authorization header
```

```
  })
);
```

`origin` cannot be the wildcard `"*"` when `credentials: true`. The CORS spec forbids combining a wildcard origin with credentialed requests (cookies, HTTP auth) because it would let any website read a user's authenticated response. You must list an exact origin (or a function that validates against an allow-list of exact origins).

```
// Supporting multiple known origins (e.g. staging + production)
const allowedOrigins = [
  "http://localhost:5173",
  "https://taskapp.example.com",
];

app.use(
  cors({
    origin: (incomingOrigin, callback) => {
      if (!incomingOrigin || allowedOrigins.includes(incomingOrigin)) {
        callback(null, true);
      } else {
        callback(new Error("Not allowed by CORS"));
      }
    },
    credentials: true,
  })
);
```

26.3 Common CORS Errors and Fixes

In production, read the allowed origin from an environment variable (`process.env.CLIENT_URL`) instead of hardcoding `localhost:5173`, so the same code works across development, staging, and production without edits.

Error in browser console	Fix
"No `Access-Control-Allow-Origin` header is present on the requested resource"	The server never sent CORS headers at all. Add `app.use(cors({...}))` before your routes, and make sure it runs for every request (including `OPTIONS`).
"The value of the `Access-Control-Allow-Origin` header in the response must not be the wildcard `*` when the request's credentials mode is `include`"	Replace `origin: "*"` with the exact frontend origin string, and keep `credentials: true` only if you actually need cookies/auth headers sent cross-origin.
"The request's credentials mode is `include`, but the `Access-Control-Allow-Credentials` header … is not `true`"	Add `credentials: true` inside the `cors()` config on the server *and* set `withCredentials: true` (axios) or `credentials: "include"` (fetch) on the frontend request.
"Response to preflight request doesn't pass access control check: It does not have HTTP ok status"	Something (an auth middleware, a rate limiter) is intercepting the `OPTIONS` request before `cors()` can respond to it. Register `cors()` as the very first middleware, ahead of auth checks.
"Method `PATCH` is not allowed by `Access-Control-Allow-Methods` in preflight response"	Pass an explicit `methods` array to `cors()`, e.g. `methods: ["GET","POST","PUT","PATCH","DELETE"]`, if you rely on methods beyond the library's defaults.

CHAPTER 27

File Uploads (Multer + Cloudinary)

27.1 The Upload Pipeline

Uploading a profile avatar or a task attachment involves three separate concerns: getting the raw file bytes off the HTTP request (Multer), storing them somewhere durable and CDN-backed (Cloudinary), and saving the resulting URL on a MongoDB document so the rest of the app can reference it like any other field.

Upload Pipeline

```
React (file input) --> FormData --> POST multipart/form-data
        |
        v
   Multer middleware (parses multipart body, exposes req.file)
        |
        v
   Cloudinary (cloudinary.uploader.upload)
        |
        v
   secure_url returned
        |
        v
   Save url on User/Task document in MongoDB
```

27.1.1 Why multipart/form-data

JSON bodies (`application/json`) can only carry text. Binary file data is sent using the `multipart/form-data` encoding, which packs each form field — text or file — into its own labeled part of the request body. `express.json()` cannot parse this format at all, which is why a dedicated middleware (Multer) is required specifically for routes that accept files.

27.2 Backend: Multer Configuration

Multer can buffer files in memory or write them to disk. Since the buffer is being immediately forwarded to Cloudinary rather than kept locally, memory storage avoids an unnecessary disk write.

```
// backend/src/middleware/upload.middleware.js
const multer = require("multer");

const storage = multer.memoryStorage();

const fileFilter = (req, file, cb) => {
  const allowedMimeTypes = ["image/jpeg", "image/png", "image/webp"];
  if (allowedMimeTypes.includes(file.mimetype)) {
    cb(null, true);
  } else {
    cb(new Error("Only JPEG, PNG, or WEBP images are allowed"), false);
```

```
  }
};

const upload = multer({
  storage,
  fileFilter,
  limits: { fileSize: 2 * 1024 * 1024 }, // 2 MB max
});

module.exports = upload;
```

27.2.1 Cloudinary Configuration

```
// backend/src/config/cloudinary.js
const cloudinary = require("cloudinary").v2;

cloudinary.config({
  cloud_name: process.env.CLOUDINARY_CLOUD_NAME,
  api_key: process.env.CLOUDINARY_API_KEY,
  api_secret: process.env.CLOUDINARY_API_SECRET,
});

module.exports = cloudinary;
```

`CLOUDINARY_CLOUD_NAME`, `CLOUDINARY_API_KEY`, and `CLOUDINARY_API_SECRET` come from the Cloudinary dashboard and belong in `.env`, never committed to source control (see Chapter 24 on environment configuration).

27.2.2 Controller: Upload and Save

```
// backend/src/controllers/user.controller.js
const cloudinary = require("../config/cloudinary");
const User = require("../models/User");

const uploadAvatar = async (req, res, next) => {
  try {
    if (!req.file) {
      return res.status(400).json({ message: "No image file provided" });
    }

    // Convert the in-memory buffer to a data URI Cloudinary can accept
    const base64 = req.file.buffer.toString("base64");
    const dataUri = `data:${req.file.mimetype};base64,${base64}`;

    const result = await cloudinary.uploader.upload(dataUri, {
      folder: "task-app/avatars",
      resource_type: "image",
    });

    const user = await User.findByIdAndUpdate(
      req.user.id,
      { avatar: result.secure_url },
      { new: true }
    ).select("-password");
```

```
    res.status(200).json({ user });
  } catch (error) {
    next(error);
  }
};

module.exports = { uploadAvatar };
```

27.2.3 Route Wiring

```
// backend/src/routes/user.routes.js
const express = require("express");
const upload = require("../middleware/upload.middleware");
const { uploadAvatar } = require("../controllers/user.controller");
const { protect } = require("../middleware/auth.middleware");

const router = express.Router();

router.post("/me/avatar", protect, upload.single("avatar"), uploadAvatar);

module.exports = router;
```

The field name passed to `upload.single("avatar")` must exactly match the key the frontend appends to `FormData`. A mismatch results in `req.file` being `undefined` with no error thrown.

27.3 Frontend: Building and Sending FormData

```
// frontend/src/components/AvatarUploader.jsx
import { useState } from "react";
import api from "../services/api";

function AvatarUploader({ onUploaded }) {
  const [file, setFile] = useState(null);
  const [uploading, setUploading] = useState(false);

  const handleSubmit = async (e) => {
    e.preventDefault();
    if (!file) return;

    const formData = new FormData();
    formData.append("avatar", file); // key MUST match upload.single("avatar")

    setUploading(true);
    try {
      const { data } = await api.post("/users/me/avatar", formData, {
        headers: { "Content-Type": "multipart/form-data" },
      });
      onUploaded(data.user.avatar);
    } catch (error) {
      console.error("Upload failed:", error.response?.data?.message);
    } finally {
      setUploading(false);
```

```
    }
  };

  return (
    <form onSubmit={handleSubmit}>
      <input
        type="file"
        accept="image/png, image/jpeg, image/webp"
        onChange={(e) => setFile(e.target.files[0])}
      />
      <button type="submit" disabled={!file || uploading}>
        {uploading ? "Uploading..." : "Upload Avatar"}
      </button>
    </form>
  );
}

export default AvatarUploader;
```

Axios will set the correct `multipart/form-data` boundary automatically when you pass a `FormData` instance as the body – explicitly setting the `Content-Type` header is optional in most setups, but harmless to include for clarity.

CHAPTER 28

Search, Filter, and Sort

28.1 Target Endpoint

```
GET /api/tasks?page=1&limit=10&search=mern&status=completed&sort=createdAt
```

Query Param	Purpose
`page`	Which page of results to return (used with `limit`, see Chapter 29).
`limit`	How many documents per page.
`search`	Free-text term matched against the task title (and optionally description).
`status`	Exact-match filter, e.g. `pending` / `in-progress` / `completed`.
`sort`	Field to sort by; a leading `-` means descending (e.g. `-createdAt`).

28.2 Backend: Building the Filter Dynamically

The controller builds a plain filter object incrementally, only adding keys for parameters the client actually sent. This keeps the query flexible without needing a different code path per combination of filters.

```
// backend/src/controllers/task.controller.js
const Task = require("../models/Task");

const getTasks = async (req, res, next) => {
  try {
    const { search, status, sort } = req.query;

    // ---- Build the filter object ----
    const filter = { owner: req.user.id };

    if (status) {
      filter.status = status; // exact match
    }

    if (search) {
      // Approach A: regex search (works without a text index, case-insensitive)
      filter.title = { $regex: search, $options: "i" };

      // Approach B (alternative): use a MongoDB text index instead:
      // taskSchema.index({ title: "text", description: "text" });
      // filter.$text = { $search: search };
      // $text is faster on large collections but requires the index above
      // and does not support partial-word matching the way $regex does.
    }

    // ---- Build the sort object ----
    // "sort=-createdAt" -> { createdAt: -1 }; "sort=title" -> { title: 1 }
```

```
    let sortObj = { createdAt: -1 }; // default: newest first
    if (sort) {
      const direction = sort.startsWith("-") ? -1 : 1;
      const field = sort.replace(/^-/, "");
      sortObj = { [field]: direction };
    }

    const tasks = await Task.find(filter).sort(sortObj);

    res.status(200).json({ data: tasks });
  } catch (error) {
    next(error);
  }
};

module.exports = { getTasks };
```

This chapter combines search/filter/sort in isolation. Chapter 29 folds `.skip()` and `.limit()` into the same query so all four concerns — search, filter, sort, and pagination — work together in one request.

28.3 Frontend: Query State and Service Function

```
// frontend/src/services/taskService.js
import api from "./api";

export const fetchTasks = async ({ search, status, sort }) => {
  const params = new URLSearchParams();
  if (search) params.append("search", search);
  if (status) params.append("status", status);
  if (sort) params.append("sort", sort);

  const { data } = await api.get(`/tasks?${params.toString()}`);
  return data.data;
};
```

```
// frontend/src/components/TaskFilters.jsx
import { useState, useEffect } from "react";
import { fetchTasks } from "../services/taskService";

function TaskFilters({ onResults }) {
  const [search, setSearch] = useState("");
  const [status, setStatus] = useState("");
  const [sort, setSort] = useState("-createdAt");

  useEffect(() => {
    const timeoutId = setTimeout(async () => {
      const tasks = await fetchTasks({ search, status, sort });
      onResults(tasks);
    }, 400); // debounce so we don't fire a request on every keystroke

    return () => clearTimeout(timeoutId);
  }, [search, status, sort, onResults]);
```

```
  return (
    <div>
      <input
        type="text"
        placeholder="Search tasks..."
        value={search}
        onChange={(e) => setSearch(e.target.value)}
      />

      <select value={status} onChange={(e) => setStatus(e.target.value)}>
        <option value="">All statuses</option>
        <option value="pending">Pending</option>
        <option value="in-progress">In Progress</option>
        <option value="completed">Completed</option>
      </select>

      <select value={sort} onChange={(e) => setSort(e.target.value)}>
        <option value="-createdAt">Newest first</option>
        <option value="createdAt">Oldest first</option>
        <option value="title">Title (A-Z)</option>
        <option value="-title">Title (Z-A)</option>
      </select>
    </div>
  );
}

export default TaskFilters;
```

Debouncing the search input (waiting 400ms after the last keystroke before firing the request) avoids sending a network request per character typed.

CHAPTER 29

Pagination

29.1 The skip/limit Formula

Returning every matching document in one response doesn't scale once a user has hundreds of tasks. Pagination splits results into pages using two numbers derived from the requested page: how many documents to skip, and how many to return.

```
const page = Number(req.query.page) || 1;
const limit = Number(req.query.limit) || 10;
const skip = (page - 1) * limit;
```

For page 1 with a limit of 10, `skip` is 0 — return the first 10 documents. For page 2, `skip` is 10 — skip the first 10 and return the next 10. And so on.

Pagination Flow

```
Frontend requests page=2, limit=10
        |
        v
   GET /api/tasks?page=2&limit=10
        |
        v
   skip = (2 - 1) * 10 = 10
        |
        v
   MongoDB: Task.find(filter).skip(10).limit(10)
        |
        v
   Response: { data, page: 2, limit: 10, total, totalPages }
```

29.2 Backend Implementation

Getting the total document count alongside the page of results lets the frontend render page numbers and disable "Next" on the last page.

```
// backend/src/controllers/task.controller.js
const Task = require("../models/Task");

const getTasks = async (req, res, next) => {
  try {
    const page = Number(req.query.page) || 1;
    const limit = Number(req.query.limit) || 10;
    const skip = (page - 1) * limit;

    const filter = { owner: req.user.id };
    if (req.query.status) filter.status = req.query.status;
    if (req.query.search) {
      filter.title = { $regex: req.query.search, $options: "i" };
```

```
    }

    const sortObj = { createdAt: -1 };

    const [tasks, total] = await Promise.all([
      Task.find(filter).sort(sortObj).skip(skip).limit(limit),
      Task.countDocuments(filter),
    ]);

    const totalPages = Math.ceil(total / limit);

    res.status(200).json({
      data: tasks,
      page,
      limit,
      total,
      totalPages,
    });
  } catch (error) {
    next(error);
  }
};

module.exports = { getTasks };
```

`Task.countDocuments(filter)` must use the *same* filter object as the `find()` call, otherwise `total` and `totalPages` won't correspond to the actual filtered result set.

29.2.1 Response Shape

This is the exact shape referenced by the API reference table in Chapter 8 — every paginated list endpoint in the Task Management System follows it consistently.

```
{
  "data": [ /* array of up to `limit` task objects */ ],
  "page": 2,
  "limit": 10,
  "total": 45,
  "totalPages": 5
}
```

29.3 Frontend: Pagination Controls

```
// frontend/src/components/Pagination.jsx
function Pagination({ page, totalPages, onPageChange }) {
  const pageNumbers = Array.from({ length: totalPages }, (_, i) => i + 1);

  return (
    <div className="pagination">
      <button disabled={page <= 1} onClick={() => onPageChange(page - 1)}>
        Prev
      </button>

      {pageNumbers.map((num) => (
```

```
        <button
          key={num}
          className={num === page ? "active" : ""}
          onClick={() => onPageChange(num)}
        >
          {num}
        </button>
      ))}

      <button
        disabled={page >= totalPages}
        onClick={() => onPageChange(page + 1)}
      >
        Next
      </button>
    </div>
  );
}

export default Pagination;
```

```
// frontend/src/pages/TaskList.jsx
import { useState, useEffect } from "react";
import api from "../services/api";
import Pagination from "../components/Pagination";

function TaskList() {
  const [tasks, setTasks] = useState([]);
  const [page, setPage] = useState(1);
  const [totalPages, setTotalPages] = useState(1);

  useEffect(() => {
    const load = async () => {
      const { data } = await api.get(`/tasks?page=${page}&limit=10`);
      setTasks(data.data);
      setTotalPages(data.totalPages);
    };
    load();
  }, [page]);

  return (
    <div>
      {tasks.map((task) => (
        <div key={task._id}>{task.title}</div>
      ))}
      <Pagination page={page} totalPages={totalPages} onPageChange={setPage} />
    </div>
  );
}

export default TaskList;
```

Keep `page` in the URL's query string (e.g. via `useSearchParams` from React Router) so a reload or a shared link lands on the same page instead of always resetting to page 1.

CHAPTER 30

MongoDB Indexing

30.1 What an Index Actually Does

Without an index, MongoDB has to perform a **collection scan** — check every single document against the query filter, one by one. An index is a separate, ordered data structure (conceptually similar to a book's index) that maps field values directly to the documents that contain them, so MongoDB can jump straight to matches instead of scanning everything.

Without an Index

```
Query: { status: "completed" }
        |
        v
   Scan document 1 -> check status -> no match
   Scan document 2 -> check status -> no match
   Scan document 3 -> check status -> match!
   ... (continues for every document in the collection)
```

With an Index

```
Query: { status: "completed" }
        |
        v
   Look up "completed" in the status index
        |
        v
   Index points directly at matching documents
        |
        v
   Return matches (no scan of unrelated documents)
```

30.2 Unique Indexes

A unique index both speeds up lookups *and* enforces that no two documents can share the same value for that field. Declaring `unique: true` on a schema field creates this index automatically.

```
// backend/src/models/User.js
const userSchema = new mongoose.Schema({
  email: {
    type: String,
    required: true,
    unique: true, // creates a unique index on `email`
    lowercase: true,
    trim: true,
  },
  password: { type: String, required: true },
});
```

A unique index is what actually causes the duplicate-email `E11000 duplicate key error` at the database layer, which your error handler can translate into a clean `409 Conflict` response (see Chapter 25).

30.3 Compound Indexes

A compound index covers multiple fields together, and is especially valuable when a query filters on one field and sorts on another — exactly the search/filter/sort pattern from Chapter 28.

```
// backend/src/models/Task.js
const taskSchema = new mongoose.Schema({
  title: { type: String, required: true },
  status: { type: String, enum: ["pending", "in-progress", "completed"] },
  owner: { type: mongoose.Schema.Types.ObjectId, ref: "User" },
}, { timestamps: true });

// Speeds up: Task.find({ status: "completed" }).sort({ createdAt: -1 })
taskSchema.index({ status: 1, createdAt: -1 });

module.exports = mongoose.model("Task", taskSchema);
```

The order of fields in a compound index matters: this index efficiently supports queries that filter on `status` alone, or on `status` + sort by `createdAt`, but is less useful for a query that sorts by `createdAt` without filtering on `status` at all.

30.4 When to Add an Index

Add an index when a field is frequently used to **filter** (`status`, `owner`), **sort** (`createdAt`), or **look up uniquely** (`email`). These are exactly the fields that appear in `.find()`, `.sort()`, or lookup queries across the app's most common requests.

Indexes are not free. Every index has to be updated on every insert, update, and delete, which slows down writes, and every index consumes additional disk and memory. Don't index every field "just in case" — only index fields that are actually queried or sorted on frequently, and periodically review indexes on collections that see heavy write traffic.

CHAPTER 31

MongoDB Aggregation

31.1 The Pipeline Concept

An aggregation pipeline processes documents through a sequence of stages, each one transforming the output of the previous stage. Unlike `.find()`, which returns matching documents as-is, aggregation can reshape, group, and compute across documents — exactly what's needed for dashboard statistics.

Aggregation Pipeline

```
Task collection
      |
      v
  $match   --> filter documents (like a WHERE clause)
      |
      v
  $group   --> bucket documents and compute per-bucket values
      |
      v
  $sort    --> order the grouped results
      |
      v
  $project --> reshape the output fields
      |
      v
  Final result array
```

31.2 Dashboard Stats Endpoint

`GET /api/tasks/stats` needs: total task count, completed count, pending count, and a breakdown of tasks per user. This maps to two separate `$group` stages in one pipeline.

```
// backend/src/controllers/task.controller.js
const Task = require("../models/Task");

const getTaskStats = async (req, res, next) => {
  try {
    const statusCounts = await Task.aggregate([
      // Stage 1: only this user's tasks (skip if stats are app-wide)
      { $match: { owner: req.user.id } },

      // Stage 2: group by status, counting documents in each group
      {
        $group: {
          _id: "$status",
          count: { $sum: 1 },
        },
```

```
    },

    // Stage 3: alphabetical by status for a stable response order
    { $sort: { _id: 1 } },
  ]);

  // Reshape [{ _id: "completed", count: 5 }, ...] into a flat object
  const summary = { total: 0, completed: 0, pending: 0, "in-progress": 0 };
  statusCounts.forEach(({ _id, count }) => {
    summary[_id] = count;
    summary.total += count;
  });

  const perUser = await Task.aggregate([
    {
      $group: {
        _id: "$owner",
        taskCount: { $sum: 1 },
      },
    },
    // Join back to the users collection to attach a readable name
    {
      $lookup: {
        from: "users",
        localField: "_id",
        foreignField: "_id",
        as: "user",
      },
    },
    { $unwind: "$user" },
    {
      $project: {
        _id: 0,
        userId: "$_id",
        name: "$user.name",
        taskCount: 1,
      },
    },
    { $sort: { taskCount: -1 } },
  ]);

  res.status(200).json({ summary, perUser });
 } catch (error) {
  next(error);
 }
};

module.exports = { getTaskStats };
```

`$lookup` performs a join against another collection — here, pulling in each owner's `name` from the `users` collection so the response doesn't just show raw `ObjectIds`. It plays the same role that `.populate()` plays in a normal Mongoose query, but inside an aggregation pipeline `.populate()` itself isn't available, so `$lookup` is the aggregation-native equivalent.

31.2.1 Example Output

```
{
  "summary": {
    "total": 45,
    "completed": 20,
    "pending": 15,
    "in-progress": 10
  },
  "perUser": [
    { "userId": "64f...a1", "name": "Asha Patel", "taskCount": 18 },
    { "userId": "64f...b2", "name": "Ravi Kumar", "taskCount": 15 },
    { "userId": "64f...c3", "name": "Meera Nair", "taskCount": 12 }
  ]
}
```

31.3 Route Wiring

```
// backend/src/routes/task.routes.js
router.get("/stats", protect, getTaskStats);
```

Aggregation stages run in the order you list them, and `$match` stages placed early narrow down the working set before the more expensive `$group`/`$lookup` stages run — put filters as close to the top of the pipeline as possible.

CHAPTER 32

Frontend Auth State (AuthContext)

32.1 Why a Context Instead of Prop Drilling

Whether a user is logged in, and who they are, is needed by components scattered across the whole tree — the navbar shows their name, protected routes need to check they're logged in at all, and profile pages need their data. Passing that down as props through every intermediate component would mean threading auth state through components that don't care about it at all. React's Context API lets any component subscribe directly to auth state without that chain.

AuthProvider Tree

```
                <AuthProvider>
                       |
        ----------------------------------
        |            |             |      |
   <Navbar>   <Dashboard>    <Profile>  <ProtectedRoute>
    (shows      (shows         (shows     (checks
    user.name)  user's tasks)  user data) currentUser)
```

32.2 AuthContext Implementation

```jsx
// frontend/src/context/AuthContext.jsx
import { createContext, useContext, useState, useEffect } from "react";
import api from "../services/api";

const AuthContext = createContext(null);

export function AuthProvider({ children }) {
  const [currentUser, setCurrentUser] = useState(null);
  const [loading, setLoading] = useState(true);

  // Restore session on app load by asking the backend who the
  // current cookie/token belongs to, if anyone.
  useEffect(() => {
    const checkAuth = async () => {
      try {
        const { data } = await api.get("/auth/me");
        setCurrentUser(data.user);
      } catch {
        setCurrentUser(null);
      } finally {
        setLoading(false);
      }
    };
    checkAuth();
  }, []);

  const login = async (credentials) => {
```

```
    const { data } = await api.post("/auth/login", credentials);
    setCurrentUser(data.user);
    return data.user;
  };

  const logout = async () => {
    await api.post("/auth/logout");
    setCurrentUser(null);
  };

  const value = { currentUser, loading, login, logout };

  return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>;
}

export function useAuth() {
  const context = useContext(AuthContext);
  if (!context) {
    throw new Error("useAuth must be used within an AuthProvider");
  }
  return context;
}
```

`loading` exists so consumers (especially `ProtectedRoute`) can distinguish "we don't know yet whether the user is logged in" from "we checked, and they are not." Without it, a protected route would briefly redirect to `/login` on every page refresh before `/auth/me` has had a chance to respond.

32.3 Wiring the Provider in main.jsx

```
// frontend/src/main.jsx
import React from "react";
import ReactDOM from "react-dom/client";
import { BrowserRouter } from "react-router-dom";
import App from "./App";
import { AuthProvider } from "./context/AuthContext";

ReactDOM.createRoot(document.getElementById("root")).render(
  <React.StrictMode>
    <BrowserRouter>
      <AuthProvider>
        <App />
      </AuthProvider>
    </BrowserRouter>
  </React.StrictMode>
);
```

32.4 Consuming the Context: Navbar

```
// frontend/src/components/Navbar.jsx
import { useNavigate } from "react-router-dom";
import { useAuth } from "../context/AuthContext";
```

```
function Navbar() {
  const { currentUser, logout } = useAuth();
  const navigate = useNavigate();

  const handleLogout = async () => {
    await logout();
    navigate("/login");
  };

  return (
    <nav>
      <span className="brand">Task Manager</span>
      {currentUser ? (
        <div className="navbar-user">
          <span>Hi, {currentUser.name}</span>
          <button onClick={handleLogout}>Logout</button>
        </div>
      ) : (
        <a href="/login">Login</a>
      )}
    </nav>
  );
}

export default Navbar;
```

Every component that needs auth state — `Navbar`, `ProtectedRoute`, `Profile` — calls the same `useAuth()` hook instead of importing `AuthContext` directly. It's a small wrapper, but it gives a single place to add the "used outside provider" guard and keeps consumers decoupled from the context's internal shape.

CHAPTER 33

API Loading and Error States

Every component that fetches data from an API passes through the same handful of states, whether it is a task list, a user profile, or a dashboard widget. Handling these states consistently is what separates an app that feels broken during a slow network from one that feels solid.

33.1 The State Chain

At minimum, a data-fetching component moves through `idle`, `loading`, and then either `success` or `error`. A well-behaved component never leaves the user staring at a blank screen or a frozen spinner.

API Component State Chain

```
   idle
    |
    |  useEffect fires -> fetch starts
    v
 loading  ------------------------> (spinner / skeleton shown)
    |
    +----------------------+
    |                      |
    v                      v
 success                error
    |                      |
    v                      v
 data.length === 0?   show error message
    |         |            + Retry button
  yes        no                 |
    |         |                 v
    v         v           user clicks Retry
 empty state  render list  ---> back to loading
```

There are really four visible outcomes, not two: **loading**, **error**, **empty** (success but no data), and **populated** (success with data). Most bugs in real apps come from forgetting the empty state and assuming success always means "there is a list to render."

33.2 Implementing the Pattern

The same three pieces of state — data, loading, error — cover almost every fetching component in the app.

```jsx
// components/TaskList.jsx
import { useEffect, useState } from "react";
import api from "../api/axios";
```

```
function TaskList() {
  const [tasks, setTasks] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  const fetchTasks = async () => {
    setLoading(true);
    setError(null);
    try {
      const res = await api.get("/tasks");
      setTasks(res.data.data);
    } catch (err) {
      setError(err.response?.data?.message || "Failed to load tasks");
    } finally {
      setLoading(false);
    }
  };

  useEffect(() => {
    fetchTasks();
  }, []);

  if (loading) return <Spinner />;

  if (error) {
    return (
      <div className="text-center py-8">
        <p className="text-red-600 mb-3">{error}</p>
        <button onClick={fetchTasks} className="px-4 py-2 bg-blue-600 text-white
            rounded">
          Retry
        </button>
      </div>
    );
  }

  if (!tasks.length) {
    return <p className="text-gray-500 text-center py-8">No tasks yet. Create your
        first one!</p>;
  }

  return (
    <ul className="space-y-2">
      {tasks.map((task) => (
        <li key={task._id}>{task.title}</li>
      ))}
    </ul>
  );
}

export default TaskList;
```

Always reset `error` to `null` at the start of a new fetch attempt. Without it, a successful retry can leave a stale error message rendered alongside (or instead of) the fresh data, because `error` never got cleared.

33.3 Loading Spinners

A spinner is the simplest loading indicator — appropriate for fast, small requests where showing structure would be overkill.

```
// components/Spinner.jsx
function Spinner() {
  return (
    <div className="flex justify-center items-center py-8">
      <div className="h-8 w-8 border-4 border-blue-600 border-t-transparent
                  rounded-full animate-spin" />
    </div>
  );
}

export default Spinner;
```

33.4 Skeleton Screens

For content-heavy views (task cards, profile pages), a skeleton that mimics the eventual layout reduces perceived loading time better than a spinner, because the user sees the shape of what's coming.

```
// components/TaskCardSkeleton.jsx
function TaskCardSkeleton() {
  return (
    <div className="border rounded-lg p-4 space-y-3 animate-pulse">
      <div className="h-4 bg-gray-200 rounded w-3/4" />
      <div className="h-3 bg-gray-200 rounded w-1/2" />
      <div className="h-3 bg-gray-200 rounded w-full" />
    </div>
  );
}

// Usage while loading:
// {loading
// ? Array.from({ length: 4 }).map((_, i) => <TaskCardSkeleton key={i} />)
// : tasks.map((t) => <TaskCard key={t._id} task={t} />)}
```

Use spinners for quick actions (button clicks, form submits) and skeletons for initial page/list loads. A spinner on a full list load makes the whole page flash blank-then-full; a skeleton feels smoother because the layout never jumps.

33.5 Choosing the Right State to Show

Situation	What to Render
Request in flight	Spinner (short lists/actions) or skeleton (page loads)
Request failed	Error message + Retry button, never a blank screen
Request succeeded, empty array	Friendly empty state with a call to action (e.g. ”Create your first task”)
Request succeeded, has data	The actual list/content

WHAT: Every API-driven component tracks `loading`, `error`, and `data` state and renders a different UI for each combination.

WHY: Without explicit states, slow networks show blank screens, failed requests show nothing or crash the app, and empty results look indistinguishable from a bug.

HOW: Wrap the fetch in try/catch/finally, set the three state variables accordingly, and branch the render with early returns: loading first, then error (with Retry), then empty, then the populated view.

CHAPTER 34

Forms

Forms are where most of the frontend's interaction with the backend happens: login, registration, and every create/edit screen for tasks. This chapter covers the practical pattern used across all of them — controlled inputs backed by a single state object, validated before submit, and wired to the API with proper loading/error handling.

34.1 Controlled Inputs and a Single Form State Object

A controlled input's value is driven entirely by React state, and every keystroke updates that state through `onChange`. Rather than one `useState` per field, keep the whole form as a single object and use one generic handler keyed by the input's `name` attribute.

```
const [form, setForm] = useState({ email: "", password: "" });

const handleChange = (e) => {
  setForm({ ...form, [e.target.name]: e.target.value });
};

// <input name="email" value={form.email} onChange={handleChange} />
// <input name="password" value={form.password} onChange={handleChange} />
```

This one handler works for any number of fields as long as each input's `name` matches a key in the form object. Add a field to the form by adding a key to the initial state and an input with a matching `name` — no new handler needed.

34.2 Example 1: LoginForm

```
// components/LoginForm.jsx
import { useState } from "react";
import api from "../api/axios";

function LoginForm({ onSuccess }) {
  const [form, setForm] = useState({ email: "", password: "" });
  const [errors, setErrors] = useState({});
  const [loading, setLoading] = useState(false);
  const [serverError, setServerError] = useState("");

  const handleChange = (e) => {
    setForm({ ...form, [e.target.name]: e.target.value });
  };

  const validate = () => {
    const errs = {};
    if (!form.email) errs.email = "Email is required";
    if (!form.password) errs.password = "Password is required";
```

```
    return errs;
  };

  const handleSubmit = async (e) => {
    e.preventDefault();
    const errs = validate();
    setErrors(errs);
    if (Object.keys(errs).length) return;

    setLoading(true);
    setServerError("");
    try {
      const res = await api.post("/auth/login", form);
      onSuccess(res.data.data);
      setForm({ email: "", password: "" });
    } catch (err) {
      setServerError(err.response?.data?.message || "Login failed");
    } finally {
      setLoading(false);
    }
  };

  return (
    <form onSubmit={handleSubmit} className="space-y-4">
      <div>
        <input name="email" value={form.email} onChange={handleChange} placeholder="
            Email" />
        {errors.email && <p className="text-red-600 text-sm">{errors.email}</p>}
      </div>
      <div>
        <input name="password" type="password" value={form.password} onChange={
            handleChange} placeholder="Password" />
        {errors.password && <p className="text-red-600 text-sm">{errors.password}</p>}
      </div>
      {serverError && <p className="text-red-600 text-sm">{serverError}</p>}
      <button type="submit" disabled={loading}>
        {loading ? "Logging in..." : "Log In"}
      </button>
    </form>
  );
}

export default LoginForm;
```

34.3 Example 2: RegisterForm with Confirm Password

```
// components/RegisterForm.jsx
import { useState } from "react";
import api from "../api/axios";

function RegisterForm() {
  const [form, setForm] = useState({
    name: "", email: "", password: "", confirmPassword: "",
  });
  const [errors, setErrors] = useState({});
  const [loading, setLoading] = useState(false);
```

```
  const handleChange = (e) => {
    setForm({ ...form, [e.target.name]: e.target.value });
  };

  const validate = () => {
    const errs = {};
    if (!form.name) errs.name = "Name is required";
    if (!/^\S+@\S+\.\S+$/.test(form.email)) errs.email = "Enter a valid email";
    if (form.password.length < 6) errs.password = "Password must be at least 6
        characters";
    if (form.confirmPassword !== form.password) errs.confirmPassword = "Passwords do
        not match";
    return errs;
  };

  const handleSubmit = async (e) => {
    e.preventDefault();
    const errs = validate();
    setErrors(errs);
    if (Object.keys(errs).length) return;

    setLoading(true);
    try {
      const { confirmPassword, ...payload } = form;
      await api.post("/auth/register", payload);
      setForm({ name: "", email: "", password: "", confirmPassword: "" });
      setErrors({});
    } catch (err) {
      setErrors({ form: err.response?.data?.message || "Registration failed" });
    } finally {
      setLoading(false);
    }
  };

  return (
    <form onSubmit={handleSubmit} className="space-y-4">
      <input name="name" value={form.name} onChange={handleChange} placeholder="Name"
          />
      {errors.name && <p className="text-red-600 text-sm">{errors.name}</p>}

      <input name="email" value={form.email} onChange={handleChange} placeholder="
          Email" />
      {errors.email && <p className="text-red-600 text-sm">{errors.email}</p>}

      <input name="password" type="password" value={form.password} onChange={
          handleChange} placeholder="Password" />
      {errors.password && <p className="text-red-600 text-sm">{errors.password}</p>}

      <input name="confirmPassword" type="password" value={form.confirmPassword}
          onChange={handleChange} placeholder="Confirm Password" />
      {errors.confirmPassword && <p className="text-red-600 text-sm">{errors.
          confirmPassword}</p>}

      {errors.form && <p className="text-red-600 text-sm">{errors.form}</p>}
      <button type="submit" disabled={loading}>
        {loading ? "Creating account..." : "Register"}
      </button>
    </form>
  );
}
```

```
export default RegisterForm;
```

Never send `confirmPassword` to the backend. It exists only for client-side validation — strip it from the payload before the API call, as shown with the destructuring in `handleSubmit` above.

34.4 Example 3: TaskForm --- Create/Edit Dual Mode

The same form component handles both creating a new task and editing an existing one. The presence of an `id` prop determines the mode: if an `id` is passed, the form loads that task's data and submits a `PUT`; otherwise it starts blank and submits a `POST`.

```
// components/TaskForm.jsx
import { useState, useEffect } from "react";
import api from "../api/axios";

function TaskForm({ id, onSaved }) {
  const isEditMode = Boolean(id);
  const [form, setForm] = useState({ title: "", description: "", status: "pending" })
      ;
  const [errors, setErrors] = useState({});
  const [loading, setLoading] = useState(false);

  useEffect(() => {
    if (!isEditMode) return;
    const loadTask = async () => {
      const res = await api.get(`/tasks/${id}`);
      const { title, description, status } = res.data.data;
      setForm({ title, description, status });
    };
    loadTask();
  }, [id, isEditMode]);

  const handleChange = (e) => {
    setForm({ ...form, [e.target.name]: e.target.value });
  };

  const validate = () => {
    const errs = {};
    if (!form.title.trim()) errs.title = "Title is required";
    return errs;
  };

  const handleSubmit = async (e) => {
    e.preventDefault();
    const errs = validate();
    setErrors(errs);
    if (Object.keys(errs).length) return;

    setLoading(true);
    try {
      if (isEditMode) {
        await api.put(`/tasks/${id}`, form);
      } else {
        await api.post("/tasks", form);
```

```
        setForm({ title: "", description: "", status: "pending" });
      }
      onSaved();
    } catch (err) {
      setErrors({ form: err.response?.data?.message || "Could not save task" });
    } finally {
      setLoading(false);
    }
  };

  return (
    <form onSubmit={handleSubmit} className="space-y-4">
      <input name="title" value={form.title} onChange={handleChange} placeholder="
          Title" />
      {errors.title && <p className="text-red-600 text-sm">{errors.title}</p>}

      <textarea name="description" value={form.description} onChange={handleChange}
          placeholder="Description" />

      <select name="status" value={form.status} onChange={handleChange}>
        <option value="pending">Pending</option>
        <option value="in-progress">In Progress</option>
        <option value="done">Done</option>
      </select>

      {errors.form && <p className="text-red-600 text-sm">{errors.form}</p>}
      <button type="submit" disabled={loading}>
        {loading ? "Saving..." : isEditMode ? "Update Task" : "Create Task"}
      </button>
    </form>
  );
}

export default TaskForm;
```

This create/edit-toggle-by-prop pattern avoids writing two nearly identical form components. The parent decides the mode simply by rendering `<TaskForm />` for create or `<TaskForm id=task._id />` for edit.

34.5 Resetting Forms After Success

Always reset a create form back to its initial values after a successful submit (see `RegisterForm` and the create branch of `TaskForm` above). Edit forms typically don't need a reset — they either stay on the saved values or the parent navigates away.

WHAT: Forms use controlled inputs backed by a single state object, a generic `handleChange`, client-side validation before submit, and loading/error state around the API call.

WHY: Uncontrolled or scattered per-field state makes validation and reset logic inconsistent; skipping client-side validation wastes a round trip on errors that could be caught instantly.

HOW: Validate in a `validate()` function called from `handleSubmit` before `preventDefault`'s API call, set field errors from its return value, and reset the form object on success.

CHAPTER 35

Tailwind CSS for MERN Apps

This chapter does not teach CSS fundamentals (the box model, flexbox theory, specificity, etc.). It assumes you already know CSS and focuses only on how to apply Tailwind's utility classes to build the screens of the Task Management System quickly and consistently.

35.1 Responsive Prefixes

Tailwind applies styles at a breakpoint using a prefix: `sm:` (640px+), `md:` (768px+), `lg:` (1024px+). Unprefixed classes apply at all sizes; a prefixed class overrides it from that breakpoint up.

```
<div className="flex flex-col md:flex-row gap-4">
  {/* stacked on mobile, side-by-side from tablet up */}
</div>
```

35.2 Flex, Grid, and Spacing

```
{/* Flex: horizontal toolbar */}
<div className="flex items-center justify-between p-4">
  <h1 className="text-xl font-bold">Tasks</h1>
  <button className="px-4 py-2 bg-blue-600 text-white rounded">New Task</button>
</div>

{/* Grid: responsive task card grid */}
<div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 gap-4">
  {tasks.map((t) => <TaskCard key={t._id} task={t} />)}
</div>
```

Common spacing utilities: `p-4` (padding), `m-4` (margin), `gap-4` (flex/grid gap), `space-y-4` (vertical gap between stacked children without a flex/grid parent).

35.3 Styling Forms

```
<div>
  <label className="block text-sm font-medium text-gray-700 mb-1">Email</label>
  <input
    type="email"
    className="w-full px-3 py-2 border border-gray-300 rounded-md
               focus:outline-none focus:ring-2 focus:ring-blue-500
               focus:border-transparent"
  />
</div>
```

`focus:ring-2` and `focus:ring-blue-500` give a visible focus ring without relying on the browser's default outline — useful for consistent branding, but keep it visible for keyboard accessibility.

35.4 Buttons: Primary / Secondary / Danger

Compose a base class string and merge variant-specific classes, so every button shares padding/rounding/transiti but differs only in color.

```
// components/Button.jsx
const base = "px-4 py-2 rounded-md font-medium transition-colors disabled:opacity-50"
    ;

const variants = {
  primary: "bg-blue-600 text-white hover:bg-blue-700",
  secondary: "bg-gray-100 text-gray-800 hover:bg-gray-200",
  danger: "bg-red-600 text-white hover:bg-red-700",
};

function Button({ variant = "primary", children, ...props }) {
  return (
    <button className={`${base} ${variants[variant]}`} {...props}>
      {children}
    </button>
  );
}

export default Button;

// <Button variant="primary">Save</Button>
// <Button variant="secondary">Cancel</Button>
// <Button variant="danger">Delete</Button>
```

35.5 Cards: TaskCard

```
// components/TaskCard.jsx
function TaskCard({ task }) {
  const statusColor = {
    pending: "bg-yellow-100 text-yellow-800",
    "in-progress": "bg-blue-100 text-blue-800",
    done: "bg-green-100 text-green-800",
  };

  return (
    <div className="border border-gray-200 rounded-lg p-4 shadow-sm
                    hover:shadow-md transition-shadow bg-white">
      <div className="flex justify-between items-start mb-2">
        <h3 className="font-semibold text-gray-900">{task.title}</h3>
        <span className={`text-xs px-2 py-1 rounded-full ${statusColor[task.status]}`
            }>
          {task.status}
        </span>
      </div>
      <p className="text-sm text-gray-600 line-clamp-2">{task.description}</p>
    </div>
  );
}
```

```
export default TaskCard;
```

35.6 Modals

A modal overlay uses a fixed, full-screen backdrop with the dialog centered on top of it.

```
// components/Modal.jsx
function Modal({ isOpen, onClose, children }) {
  if (!isOpen) return null;

  return (
    <div className="fixed inset-0 bg-black/50 flex items-center justify-center z-50">
      <div className="bg-white rounded-lg shadow-xl w-full max-w-md p-6 relative">
        <button
          onClick={onClose}
          className="absolute top-3 right-3 text-gray-400 hover:text-gray-600"
        >
          &times;
        </button>
        {children}
      </div>
    </div>
  );
}

export default Modal;

// <Modal isOpen={showModal} onClose={() => setShowModal(false)}>
// <TaskForm onSaved={() => setShowModal(false)} />
// </Modal>
```

`fixed inset-0` pins the overlay to all four edges of the viewport regardless of scroll position. `bg-black/50` is Tailwind's slash opacity syntax for black at 50% opacity — no separate opacity class needed.

35.7 Navbar with Responsive Menu

```
// components/Navbar.jsx
import { useState } from "react";

function Navbar() {
  const [open, setOpen] = useState(false);

  return (
    <nav className="bg-white border-b border-gray-200 px-4 py-3">
      <div className="flex items-center justify-between">
        <span className="font-bold text-lg text-blue-600">TaskManager</span>

        {/* Desktop links */}
        <div className="hidden md:flex items-center gap-6">
          <a href="/tasks" className="text-gray-700 hover:text-blue-600">Tasks</a>
          <a href="/profile" className="text-gray-700 hover:text-blue-600">Profile</a>
```

```
      </div>

      {/* Mobile toggle */}
      <button className="md:hidden" onClick={() => setOpen(!open)}>
        Menu
      </button>
    </div>

    {/* Mobile links */}
    {open && (
      <div className="md:hidden mt-3 flex flex-col gap-2">
        <a href="/tasks" className="text-gray-700">Tasks</a>
        <a href="/profile" className="text-gray-700">Profile</a>
      </div>
    )}
  </nav>
 );
}

export default Navbar;
```

35.8 Sidebar Layout

```
<div className="flex min-h-screen">
  <aside className="hidden lg:block w-64 bg-gray-900 text-white p-4">
    {/* nav links */}
  </aside>
  <main className="flex-1 p-6 bg-gray-50">
    {/* page content */}
  </main>
</div>
```

`hidden lg:block` hides the sidebar below the `lg` breakpoint and shows it from 1024px up, so mobile users get the toolbar/nav instead of a permanent sidebar eating screen space.

Combine `hidden md:flex` (or `lg:block`) with a mobile toggle button anywhere you need "desktop shows X, mobile shows a menu button instead" — it's the single most reused responsive pattern across navbars, sidebars, and filter panels.

WHAT: Tailwind utility classes (layout, spacing, color, state variants like `focus:` and `hover:`, and responsive prefixes like `md:`) are composed directly in JSX to build buttons, cards, modals, navbars, and responsive layouts.

WHY: Utility classes let you style components without maintaining separate CSS files or fighting specificity, and responsive prefixes make one component adapt across breakpoints without media query boilerplate.

HOW: Build small reusable components (`Button`, `TaskCard`, `Modal`) with variant classNames, and combine `hidden`/`flex`/`grid` with breakpoint prefixes for layouts that adapt from mobile to desktop.

CHAPTER 36

Frontend Error Boundaries

Not every frontend error is an API error. A malformed prop, a `undefined.map()`, or a third-party component throwing during render will crash the entire React tree with a blank white screen unless something catches it. That "something" is an Error Boundary.

36.1 Render Errors vs. API Errors

Error Boundaries only catch errors thrown during **rendering**, **lifecycle methods**, and **constructors** of the component tree below them. They do *not* catch errors inside event handlers, async code (`setTimeout`, promises), or `fetch`/`axios` calls. A failed API request is handled by the try/catch and `error` state pattern from Chapter 33 — Error Boundaries are a separate, complementary safety net for unexpected render-time crashes.

What Each Mechanism Catches

```
Error type                  | Caught by
----------------------------|----------------------------
Component throws while      | Error Boundary
rendering (bad prop, null   |
access, etc.)               |
----------------------------|----------------------------
API call fails (network,    | try/catch + error state
4xx/5xx response)           | (Chapter 33)
----------------------------|----------------------------
Error thrown inside an      | Local try/catch in the
onClick/onSubmit handler    | handler itself
```

36.2 Building an ErrorBoundary

Error Boundaries must currently be class components — there is no hook equivalent for `getDerivedStateFromEr` or `componentDidCatch`.

```jsx
// components/ErrorBoundary.jsx
import { Component } from "react";

class ErrorBoundary extends Component {
  constructor(props) {
    super(props);
    this.state = { hasError: false, error: null };
  }

  static getDerivedStateFromError(error) {
    // Runs during the render phase; return new state to trigger the fallback UI.
    return { hasError: true, error };
  }
```

```
  componentDidCatch(error, errorInfo) {
    // Runs after render; safe place to log to a monitoring service.
    console.error("ErrorBoundary caught:", error, errorInfo);
  }

  handleReset = () => {
    this.setState({ hasError: false, error: null });
  };

  render() {
    if (this.state.hasError) {
      return (
        <div className="p-6 text-center">
          <h2 className="text-lg font-semibold text-red-600 mb-2">
            Something went wrong.
          </h2>
          <p className="text-sm text-gray-600 mb-4">
            {this.state.error?.message || "An unexpected error occurred."}
          </p>
          <button
            onClick={this.handleReset}
            className="px-4 py-2 bg-blue-600 text-white rounded"
          >
            Try Again
          </button>
        </div>
      );
    }

    return this.props.children;
  }
}

export default ErrorBoundary;
```

`getDerivedStateFromError` must be a static method returning the new state; it cannot perform side effects like logging. `componentDidCatch` is where you send the error to a logging service (Sentry, LogRocket, or your own endpoint) since it runs after the render phase completes.

36.3 Wrapping the App

Wrap boundaries around independent sections rather than one boundary around the entire app — that way a crash in one section (say, a chart widget) doesn't take down the navbar and the rest of the page with it.

```
// App.jsx
import { BrowserRouter, Routes, Route } from "react-router-dom";
import ErrorBoundary from "./components/ErrorBoundary";
import Navbar from "./components/Navbar";
import TaskListPage from "./pages/TaskListPage";
import ProfilePage from "./pages/ProfilePage";

function App() {
```

```
  return (
    <BrowserRouter>
      {/* Navbar has its own boundary so a nav crash doesn't hide the rest of the
          page */}
      <ErrorBoundary>
        <Navbar />
      </ErrorBoundary>

      <ErrorBoundary>
        <Routes>
          <Route path="/tasks" element={<TaskListPage />} />
          <Route path="/profile" element={<ProfilePage />} />
        </Routes>
      </ErrorBoundary>
    </BrowserRouter>
  );
}

export default App;
```

Granular boundaries (per route, or per risky widget) contain the damage of a crash to just that section. A single app-wide boundary is easier to set up but turns any unrelated bug into a full-page failure with no navigation left to escape from.

36.4 Error Boundaries Do Not Replace API Error Handling

Wrapping a component in an `ErrorBoundary` does nothing for a failed `axios` call inside its `useEffect` — that promise rejection happens outside React's render cycle, so the boundary never sees it. Every data-fetching component still needs its own `loading` / `error` / retry handling as covered in Chapter 33. Error Boundaries exist purely to stop unexpected render crashes from blanking the whole screen; they are a last line of defense, not a substitute for handling expected failure modes like a down API or a 500 response.

WHAT: A class-based ErrorBoundary uses `getDerivedStateFromError` and `componentDidCatch` to catch render/lifecycle errors in its child tree and show a fallback UI instead of a blank screen.

WHY: Uncaught render errors crash the entire React tree with no recovery path; API/async errors need separate handling because boundaries cannot see them.

HOW: Wrap independent sections of the app (navbar, route content, risky widgets) in their own `<ErrorBoundary>`, and keep using try/catch plus loading/error state (Chapter 33) for every API call.

CHAPTER 37

MERN Security

A MERN stack has attack surface on both ends: the Express API accepts arbitrary input from the network, and the React app runs in a browser you do not control. This chapter walks through the concrete threats that matter in practice and the specific mitigation for each, most of which are single middleware additions.

37.1 Cross-Site Scripting (XSS)

If user-supplied content (a task title, a comment) is rendered as raw HTML, an attacker can inject a `<script>` tag that runs in every other user's browser who views that content — stealing cookies, tokens, or performing actions as them.

React escapes text content by default, so `{task.title}` is safe. The risk appears only when you deliberately bypass that escaping.

```
// DANGEROUS -- only use with content you fully trust and have sanitized
<div dangerouslySetInnerHTML={{ __html: task.description }} />

// SAFE -- React escapes this automatically
<div>{task.description}</div>
```

If you must render HTML (e.g. a rich-text editor's output), sanitize it server-side or client-side with a library like `DOMPurify` before passing it to `dangerouslySetInnerHTML`. Never trust that "it's just our own users" makes stored content safe to render unescaped.

37.2 Cross-Site Request Forgery (CSRF)

CSRF tricks a logged-in user's browser into submitting a request to your API from another site, using the browser's automatically-attached cookies. It matters specifically because JWTs stored in HttpOnly cookies (Chapter 19) are sent automatically by the browser on every request to your domain.

Mitigations: set the auth cookie with `sameSite: "strict"` or `"lax"` (Chapter 19), which stops browsers from sending it on cross-site requests, and only accept state-changing requests (`POST/PUT/DELETE`) with a proper CORS configuration that whitelists your known frontend origin instead of `*`.

```
// server.js
import cors from "cors";

app.use(cors({
  origin: process.env.CLIENT_URL, // e.g. "https://taskmanager.app"
  credentials: true,
}));
```

37.3 NoSQL Injection

MongoDB queries built from raw request bodies are vulnerable to operator injection: instead of a string, an attacker sends an object like `{ "$gt": "" }` as the `password` field, which Mongo interprets as a query operator rather than a literal value — potentially bypassing a login check entirely.

```
// Attacker POSTs:
// { "email": "admin@example.com", "password": { "$gt": "" } }
// Without sanitization, User.findOne({ email, password }) can match
// any document where password is greater than an empty string --- i.e. everything.
```

Strip `$` and `.` keys from incoming data before it reaches a query. Combined with the schema-level validation from Chapter 23, this closes the operator-injection gap that field-type validation alone does not.

```
// server.js
import mongoSanitize from "express-mongo-sanitize";

app.use(mongoSanitize()); // strips keys starting with "$" or containing "."
```

Mongoose schema types already reject an object where a `String` is expected in many cases, but relying on that alone is fragile — explicit sanitization middleware closes the gap regardless of schema strictness.

37.4 Brute Force Login Attempts

Without a limit, an attacker can script thousands of password guesses against `/api/auth/login` per minute. Rate limiting caps how many attempts a single IP can make in a time window.

```
// middleware/rateLimiter.js
import rateLimit from "express-rate-limit";

export const loginLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 10, // 10 attempts per IP per window
  message: { success: false, message: "Too many login attempts. Try again later." },
  standardHeaders: true,
  legacyHeaders: false,
});

// routes/authRoutes.js
router.post("/login", loginLimiter, loginUser);
```

Apply a stricter limiter to `/login` and `/register` specifically, and a looser general limiter to the rest of the API. Locking down every route at login-attempt strictness would throttle legitimate heavy users of the app.

37.5 Credential Theft via XSS-Accessible Tokens

Storing a JWT in `localStorage` makes it readable by any JavaScript running on the page — including an injected XSS payload. HttpOnly cookies (Chapter 19) are invisible to `document.cookie` and

to any JavaScript, so an XSS bug alone cannot steal the token even if it slips past your escaping.

37.6 Insecure Cookies

An auth cookie sent without `secure` and `httpOnly` flags can be read by client scripts and transmitted over plain HTTP, where it can be intercepted on the network.

```
res.cookie("token", jwtToken, {
  httpOnly: true,
  secure: process.env.NODE_ENV === "production", // HTTPS only in prod
  sameSite: "strict",
  maxAge: 7 * 24 * 60 * 60 * 1000,
});
```

37.7 Common HTTP Header Hardening

`helmet` sets a collection of security-related HTTP headers (like disabling MIME sniffing and setting a baseline Content-Security-Policy) that block several classes of attack with one line.

```
// server.js
import helmet from "helmet";

app.use(helmet());
```

37.8 Exposed Secrets

Committing a `.env` file (or hardcoding a JWT secret, DB URI, or API key directly in source) puts credentials in Git history forever, even after deleting the file in a later commit. See Chapter 38 for the full environment variable setup; the essential rule is that `.env` is always in `.gitignore` and secrets are only read via `process.env`.

37.9 Malicious File Uploads

An unrestricted upload endpoint lets an attacker upload an executable script disguised as an image, or an oversized file meant to exhaust disk space. Restrict by MIME type and size at the multer/Cloudinary layer, and never trust the file extension alone.

```
// middleware/upload.js
import multer from "multer";

const upload = multer({
  limits: { fileSize: 2 * 1024 * 1024 }, // 2MB max
  fileFilter: (req, file, cb) => {
    const allowed = ["image/jpeg", "image/png", "image/webp"];
    if (!allowed.includes(file.mimetype)) {
      return cb(new Error("Only JPEG, PNG, or WEBP images are allowed"));
    }
    cb(null, true);
  },
});

export default upload;
```

37.10 Input Validation

Every mitigation above assumes untrusted input is validated at the edge before it reaches business logic. See Chapter 23 for the `express-validator`-based validation layer that rejects malformed requests (wrong types, missing fields, out-of-range values) before a controller or query ever runs.

37.11 Summary Table

Threat	Mitigation	Where
XSS	Rely on React's default escaping; sanitize any HTML before `dangerouslySetInnerHTML`	Ch. 36 (rendering)
CSRF	`sameSite` cookie flag + CORS origin whitelist	Ch. 19, Ch. 37
NoSQL injection	`express-mongo-sanitize` to strip `$/.` keys, plus schema validation	Ch. 23, Ch. 37
Brute force login	`express-rate-limit` on `/auth` routes	Ch. 37
Credential theft (XSS reading tokens)	HttpOnly cookies instead of `localStorage`	Ch. 19
Insecure cookies	`httpOnly`, `secure`, `sameSite` flags	Ch. 19
Header-based attacks	`helmet()` middleware	Ch. 37
Exposed secrets	`.env` + `.gitignore`, never hardcode	Ch. 38
Malicious uploads	MIME/type + size limits in multer `fileFilter`	Ch. 37
Malformed/invalid input	`express-validator` at the route layer	Ch. 23

WHAT: A secure MERN API layers helmet headers, rate limiting, input sanitization/validation, HttpOnly+secure+sameSite cookies, CORS origin restriction, upload restrictions, and secret management, instead of relying on any single defense.

WHY: Each threat (XSS, CSRF, NoSQL injection, brute force, credential theft, insecure cookies, exposed secrets, malicious uploads) exploits a different gap, so a single fix like "just validate input" leaves the others open.

HOW: Add `helmet()` and `mongoSanitize()` globally, apply `express-rate-limit` to auth routes, set cookies with `httpOnly/secure/sameSite`, whitelist CORS origins, restrict upload MIME types/sizes, and keep all secrets in an untracked `.env`.

CHAPTER 38

Environment Variables

Configuration that differs between machines and deployments — database URIs, API keys, secrets — does not belong in source code. Environment variables keep that configuration out of Git and let the same codebase run against different databases, keys, and URLs in development versus production.

38.1 Why Secrets Live Outside Source Code

A hardcoded database connection string or JWT secret is visible to anyone with repository access, and it stays in Git history even if deleted later. Environment variables are injected at runtime instead, so the source code only ever references `process.env.SOMETHING`, never the actual value.

38.2 Backend: `.env`

```
# .env (backend root -- NEVER commit this file)
PORT=5000
MONGO_URI=mongodb+srv://user:password@cluster.mongodb.net/taskmanager
JWT_SECRET=a-long-random-string-used-to-sign-tokens
GOOGLE_CLIENT_ID=your-google-oauth-client-id
GOOGLE_CLIENT_SECRET=your-google-oauth-client-secret
CLOUDINARY_CLOUD_NAME=your-cloud-name
CLOUDINARY_API_KEY=your-api-key
CLOUDINARY_API_SECRET=your-api-secret
```

Load it with `dotenv` at the very top of the entry file, before anything that reads `process.env`.

```
// server.js
import "dotenv/config"; // or: require("dotenv").config();
import express from "express";
import mongoose from "mongoose";

mongoose.connect(process.env.MONGO_URI);

const app = express();
app.listen(process.env.PORT, () => {
  console.log(`Server running on port ${process.env.PORT}`);
});
```

38.3 Frontend: `.env` with Vite

Vite only exposes variables to the client bundle if they are prefixed with `VITE_`. Anything without that prefix is invisible to the frontend code by design.

```
# .env (frontend root)
VITE_API_URL=http://localhost:5000/api
```

```
// api/axios.js
import axios from "axios";

const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL,
  withCredentials: true,
});

export default api;
```

Frontend environment variables are **not secret**. Vite (and every other bundler) inlines their values directly into the built JavaScript files at build time. Anyone can open browser DevTools, view the bundled source, and read every `VITE_*` value shipped to production. Never put an API secret, database credential, or private key behind a `VITE_` variable — only put values that are already meant to be public, like a base API URL or a public-facing OAuth client ID.

38.4 Backend vs. Frontend: Different Rules

	Backend (Node/Express)	Frontend (Vite/React)
Loaded via	`dotenv` package, `process.env.KEY`	Built into the bundle, `import.meta.env.KEY`
Naming rule	Any name	Must start with `VITE_` to be exposed
Visibility	Never sent to the browser	Compiled directly into public JS files
Safe to store secrets?	Yes	No – treat as public

38.5 Development vs. Production Configuration

Two common approaches, both valid:

- **File-based:** keep `.env.development` and `.env.production` side by side; the tool (Vite, or a script) picks the right one based on the run mode.
- **Platform-provided:** in hosting platforms (Render, Railway, Vercel, Heroku), set environment variables through the platform's dashboard/CLI instead of shipping any `.env` file at all — the platform injects them into the running process.

```
# .env.development
VITE_API_URL=http://localhost:5000/api

# .env.production
VITE_API_URL=https://api.taskmanager.app/api
```

In production hosting, the platform-provided environment variable approach is generally preferred over committing an `.env.production` file – it keeps production secrets (backend side) out of the repository entirely, even in a private one.

38.6 Always Gitignore .env

```
# .gitignore
.env
.env.*
!.env.example
```

Commit an `.env.example` with the same keys but placeholder values, so teammates know what variables the project needs without ever seeing real secrets.

```
# .env.example
PORT=5000
MONGO_URI=
JWT_SECRET=
GOOGLE_CLIENT_ID=
GOOGLE_CLIENT_SECRET=
CLOUDINARY_CLOUD_NAME=
CLOUDINARY_API_KEY=
CLOUDINARY_API_SECRET=
```

WHAT: Configuration and secrets are read from `process.env` (backend, via `dotenv`) or `import.meta.env` (frontend, via Vite's `VITE_` prefix) instead of being hardcoded.

WHY: Hardcoded secrets end up in Git history permanently, and frontend bundles are publicly readable, so backend and frontend variables need fundamentally different trust assumptions.

HOW: Keep an untracked `.env` per environment (or use platform-provided variables in production), commit only an `.env.example`, and never place real secrets behind a `VITE_` key.

CHAPTER 39

Testing APIs with Postman

Before wiring an endpoint up to React, test it in isolation. If a request works correctly in Postman but fails once the frontend calls it, the bug is in the frontend (or in how it calls the API); if it fails in Postman too, the bug is in the backend. This separation saves significant debugging time compared to only ever testing through the UI.

For authenticated routes, enable Postman's cookie jar (it's on by default in the desktop app) so the `HttpOnly` JWT cookie set by `/login` is automatically stored and re-sent on subsequent requests in the same collection — exactly like a browser would.

39.1 Register

POST `http://localhost:5000/api/auth/register`
Headers: `Content-Type: application/json`

```
// Body (raw, JSON)
{
  "name": "Ada Lovelace",
  "email": "ada@example.com",
  "password": "SecurePass123"
}
```

```
// Expected response - 201 Created
{
  "success": true,
  "data": { "id": "6614...", "name": "Ada Lovelace", "email": "ada@example.com" }
}
```

39.2 Login

POST `http://localhost:5000/api/auth/login`
Headers: `Content-Type: application/json`

```
// Body
{ "email": "ada@example.com", "password": "SecurePass123" }
```

Check the **Cookies** tab of the response in Postman — you should see the `token` cookie listed there. That confirms the cookie jar has it stored for reuse on the next request.

```
// Expected response - 200 OK
{ "success": true, "data": { "id": "6614...", "email": "ada@example.com" } }
```

39.3 Get Profile (/me)

GET `http://localhost:5000/api/auth/me`
No body needed. No manual header needed either — Postman automatically resends the stored `token` cookie from the login step.

```
// Expected response - 200 OK
{ "success": true, "data": { "id": "6614...", "name": "Ada Lovelace", "email": "
    ada@example.com" } }
```

If this returns `401 Unauthorized`, either the cookie jar did not persist the cookie (check Postman settings) or the login step did not actually set it (check the backend's `res.cookie` call).

39.4 Create Task

POST `http://localhost:5000/api/tasks`
Headers: `Content-Type: application/json`

```
// Body
{ "title": "Write chapter 39", "description": "Postman testing", "status": "pending"
    }
```

```
// Expected response - 201 Created
{
  "success": true,
  "data": { "_id": "6715...", "title": "Write chapter 39", "status": "pending", "
     owner": "6614..." }
}
```

39.5 Get Tasks

GET `http://localhost:5000/api/tasks`

```
// Expected response - 200 OK
{
  "success": true,
  "data": [
    { "_id": "6715...", "title": "Write chapter 39", "status": "pending" }
  ]
}
```

39.6 Update Task

PUT `http://localhost:5000/api/tasks/6715...`
Headers: `Content-Type: application/json`

```
// Body
{ "status": "done" }
```

```
// Expected response - 200 OK
{ "success": true, "data": { "_id": "6715...", "title": "Write chapter 39", "status":
     "done" } }
```

39.7 Delete Task

DELETE `http://localhost:5000/api/tasks/6715...`
No body needed.

```
// Expected response - 200 OK
{ "success": true, "message": "Task deleted" }
```

39.8 What to Check on Every Request

- ☐ Status code matches what the route is documented to return (200/201 for success, 4xx for expected client errors)
- ☐ Response body has the expected shape (`success`, `data`/`message` keys)
- ☐ `Content-Type: application/json` header is set on requests with a body
- ☐ Cookie jar shows the `token` cookie after login, and it's being resent on later requests
- ☐ Deleting/updating an ID that does not exist returns `404`, not a `500` crash

Save each of these requests into a Postman collection with the base URL as a collection variable (e.g. `{{baseUrl}}/tasks`). Switching from local development to a deployed backend then only requires changing one variable instead of every request.

WHAT: Every backend route is tested directly with Postman – method, URL, headers, and body – before any React code calls it, using Postman's cookie jar to persist the JWT across authenticated requests.

WHY: Testing through the UI conflates frontend and backend bugs; testing the API directly proves backend correctness first so any later frontend failure can be isolated to the frontend.

HOW: Send each request with the documented method/body, verify the status code and response shape, and confirm cookies persist across the login -> `/me` -> task CRUD sequence.

CHAPTER 40

Debugging MERN Applications

A MERN bug can live in the frontend, the network layer, the backend route, the controller logic, or the database itself. Guessing at random wastes time; checking a fixed sequence of points, in order, tells you exactly which layer the problem is in before you write a single fix.

40.1 The Core Principle

Each layer in the request/response path can only pass along what the layer before it produced. If the request body arriving at the controller is empty, no amount of controller logic will fix it — the bug is upstream, likely in how the frontend built the request. Working through the checklist in order means you stop at the first point where reality departs from what you expect, instead of debugging correct code because the actual bug is somewhere else entirely.

40.2 The Debugging Checklist

1. **Browser Console** — any thrown JS errors or warnings?
2. **Network tab** — was the request even sent?
3. **Request URL** — is it the exact endpoint you think it is?
4. **HTTP method** — `GET`/`POST`/`PUT`/`DELETE` as expected?
5. **Request body** — does it contain the data you expect, in the right shape?
6. **Headers** — is `Content-Type` (and the auth cookie) actually present?
7. **Status code** — 2xx, or does it reveal a 4xx/5xx?
8. **Response body** — what did the server actually say?
9. **Backend terminal** — any stack trace or logged error?
10. **Route** — is this request even reaching the route you expect?
11. **Controller** — is the logic doing what you think with the input it received?
12. **Database** — does the data actually look right in Compass/Atlas?

Debugging Checklist Flow

```
Browser Console
      |
      v
Network tab (was request sent?)
      |
      v
Request URL --> HTTP method --> Request body --> Headers
      |
      v
Status code --> Response body
```

```
      |
      v
Backend terminal (server-side errors/logs)
      |
      v
Route --> Controller --> Database
      |
      v
First mismatch found = the layer with the bug
```

The first six items (Console through Headers) are all frontend/network layer. Status code and response body tell you whether the backend even ran correctly. Everything from backend terminal onward is backend/database layer. Most of the checklist's value is in quickly telling you which *half* of the stack to keep investigating.

40.3 Worked Example: "Task Creation Silently Fails"

The symptom: clicking "Create Task" does nothing visible — no new task appears, no error toast, nothing.

1. **Browser Console** — no errors logged. Rules out a JS exception in the click handler.
2. **Network tab** — the `POST /api/tasks` request is there. It was sent. Rules out the button not firing at all.
3. **Request URL** — correct: `/api/tasks`.
4. **HTTP method** — correct: `POST`.
5. **Request body** — opening the request payload in DevTools shows `{}` — an empty object, even though the form clearly had a title typed into it.
6. **Headers** — checking the request headers shows `Content-Type` is missing entirely; the request was sent with the default browser content type instead of `application/json`.

Found it: the axios call omitted the JSON content-type header, so Express's `express.json()` body parser never parsed the payload, and `req.body` landed in the controller as `{}`. The fix is on the frontend, not the backend:

```
// Before -- api client missing default headers
const api = axios.create({ baseURL: import.meta.env.VITE_API_URL });

// After -- Content-Type set for every request
const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL,
  headers: { "Content-Type": "application/json" },
  withCredentials: true,
});
```

Notice the bug was found at step 6 (Headers), well before reaching the backend terminal, route, controller, or database. Jumping straight to "add console.logs in the controller" would have wasted time debugging correct backend code, since `req.body` being empty was a symptom of

a frontend request problem, not a controller bug.

Keep the browser's Network tab open with "Preserve log" enabled while reproducing a bug. It's the single fastest way to confirm whether a problem is frontend-side (request never sent, wrong URL/method/body) before you even look at the backend.

WHAT: A fixed, ordered checklist – console, network tab, URL, method, body, headers, status code, response body, backend terminal, route, controller, database – isolates which layer of the stack a bug lives in.

WHY: Without a systematic order, it's easy to debug backend logic that was never actually reached because the real problem was a malformed or unsent frontend request.

HOW: Work the checklist top to bottom and stop at the first point where behavior departs from expectation – that point is the layer containing the bug.

CHAPTER 41

Common Frontend Errors

This chapter is a reference, not a tutorial. Each entry follows the same pattern — **Cause**, **Identify**, **Solution** — so you can scan straight to the error you are seeing.

CORS Error

Cause	Backend response is missing an `Access-Control-Allow-Origin` header that matches the frontend's origin.
Identify	Console shows "has been blocked by CORS policy"; the Network tab shows the request was sent but the browser refused the response.
Solution	Add `cors({ origin: FRONTEND_URL, credentials: true })` on the backend (Chapter 26).

Network Error

Cause	The request never reached any server — backend is down, wrong port, or no internet.
Identify	Axios error has no `response` object, only `error.request`; Network tab shows the request as "failed" with no status.
Solution	Confirm the backend is running, check the URL/port, and verify `VITE_API_URL`.

404 on API Call

Cause	The frontend is calling a path that doesn't exist on the backend (typo, wrong prefix, or route not mounted).
Identify	Network tab shows status `404` with a response body (unlike Network Error, the server did respond).
Solution	Compare the exact path against `app.use('/api/...')` and the router's route definitions.

"Failed to fetch"

Cause	Generic browser message for a request that could not complete — server down, CORS preflight failure, or mixed HTTP/HTTPS content.
Identify	Thrown by `fetch`/Axios before any response is received; check the Network tab for the specific underlying reason (CORS, refused connection, etc.).
Solution	Fix the underlying cause shown in the Network tab — this message alone is not diagnostic.

"Cannot read properties of undefined (reading 'x')"

Cause	Rendering `data.x` before `data` has arrived from the API (initial state is `undefined` or `null`).
Identify	Error points to the exact line accessing the property; happens on first render, disappears after the fetch resolves.
Solution	Initialize state to a sensible default (`[]`, `null`) and guard renders with `data?.x` or an early loading return.

State Not Updating (Stale Closures)

Cause	A callback/effect captured an old value of state from a previous render and keeps using it.
Identify	Logging state inside an event handler shows the value from several renders ago, not the latest.
Solution	Use the updater-function form (`setCount(c => c + 1)`) or add the value to the effect's dependency array.

useEffect Infinite Loop

Cause	The effect updates state that is also in its own dependency array, or depends on a new object/array/function created every render.
Identify	Network tab shows the same request firing repeatedly; browser becomes sluggish.
Solution	Remove the unstable dependency, wrap it in `useMemo`/`useCallback`, or move the state update outside the effect's own trigger condition.

Duplicate API Calls

Cause	`React.StrictMode` intentionally double-invokes effects in development, or the effect is missing a dependency and re-fires on every render.
Identify	Two identical requests in the Network tab per page load; persists in dev only (StrictMode) or also in production (real bug).
Solution	If it only happens in dev, it's expected — ignore it. If it also happens in production, fix the effect's dependency array.

React Key Warning

Cause	A list rendered with `.map()` is missing a `key` prop, or two items share the same key (often `index` used after a reorder/filter).
Identify	Console warning: "Each child in a list should have a unique `key` prop."
Solution	Use a stable unique value, typically `task._id`, never the array index for lists that can reorder.

Component Not Rendering

Cause	A conditional (`if`, `&&`, early `return null`) evaluates falsy when it shouldn't, or the component isn't mounted in the tree at all.
Identify	No error in console — the component simply never appears; React DevTools shows it missing from the tree.
Solution	Log the condition's value right before the branch, and confirm the parent actually renders this child.

Wrong API URL

Cause	`VITE_API_URL` is unset (falls back to `undefined`) or has a trailing slash that doubles up with a leading slash on the request path.
Identify	Request URL in the Network tab shows `undefined/api/tasks` or `//api/tasks`.
Solution	Set `VITE_API_URL` in `.env`, keep it without a trailing slash, and build paths as `` `${baseURL}/api/tasks` ``.

Environment Variable Undefined

Cause	Vite only exposes env variables prefixed with `VITE_` to client code, or the dev server was started before the `.env` file was created/edited.
Identify	`import.meta.env.VITE_API_URL` logs `undefined` even though the variable exists in `.env`.
Solution	Rename the variable with the `VITE_` prefix and restart `npm run dev` — Vite does not hot-reload `.env` changes.

Every one of these errors is diagnosed the same way: open DevTools, check the Console tab for the message, then the Network tab for the actual request/response. Guessing without opening DevTools wastes time.

CHAPTER 42

Common Backend Errors

Same reference format as Chapter 41: **Cause**, **Identify**, **Solution** for each error, in the order they tend to appear while building the backend.

"Cannot find module 'X'"

Cause	The package isn't installed, the import path is misspelled, or a relative import is missing its file extension when using ESM.
Identify	Node throws `Error: Cannot find module` immediately on startup, with the path it tried to resolve.
Solution	Run `npm install <package>` for third-party modules, or fix the relative path for local files.

Port Already in Use (EADDRINUSE)

Cause	Another process (often a previous, still-running `nodemon` instance) is already bound to the same port.
Identify	`Error: listen EADDRINUSE: address already in use :::5000` on startup.
Solution	Kill the process on that port or change `PORT` in `.env`, then restart.

MongoDB Connection Failed

Cause	Wrong connection string, wrong credentials, or the current IP isn't on the Atlas Network Access allowlist.
Identify	`MongooseServerSelectionError` or `MongoNetworkError` shortly after `mongoose.connect()`.
Solution	Re-check `MONGO_URI`, confirm the Atlas user's password, and add your IP (or `0.0.0.0/0` for testing) to Network Access.

req.body Undefined

Cause	`express.json()` middleware isn't registered, or is registered after the routes that need it.
Identify	`req.body` is `undefined` even though the frontend sent a JSON payload (visible in the Network tab request body).
Solution	Add `app.use(express.json())` in `app.js` *before* `app.use('/api', router)`.

req.user Undefined

Cause	The `protect` middleware wasn't attached to the route, or was attached after the controller in the middleware chain.
Identify	Controller throws when reading `req.user._id` even though the request included a valid cookie/token.
Solution	Add `protect` as the route's first argument: `router.get('/me', protect, getMe)` (Chapter 20).

JWT Invalid

Cause	`JWT_SECRET` differs between signing and verifying (e.g. different `.env` on a restarted server), or the token was tampered with.
Identify	`jwt.verify()` throws `JsonWebTokenError: invalid signature`.
Solution	Ensure the same `JWT_SECRET` is used everywhere and was not rotated without invalidating old sessions.

JWT Expired

Cause	The token's `exp` claim has passed — normal behavior for a token older than its `expiresIn` setting.
Identify	`jwt.verify()` throws `TokenExpiredError`.
Solution	Respond with `401` so the frontend can redirect to login, or implement a refresh-token flow if silent renewal is needed.

Mongoose ValidationError

Cause	A document failed a schema rule — missing `required` field, value outside `enum`, wrong type, etc.
Identify	Error object has `name: 'ValidationError'` and an `errors` map keyed by field name.
Solution	Return `400` with the specific field messages; fix the payload or relax/correct the schema rule.

Duplicate Key Error (E11000)

Cause	Insert/update violates a `unique` index — most commonly a duplicate `email` on the User model.
Identify	Error message contains `E11000 duplicate key error` and the offending field/value.
Solution	Catch the error code `11000` in the controller and return a `400` like "Email already registered."

500 Internal Server Error

Cause	Generic catch-all — an unhandled exception somewhere in the request lifecycle (bad destructuring, thrown error with no specific handling, etc.).
Identify	Frontend just sees ”500”; the *real* cause is only visible in the backend terminal’s stack trace.
Solution	Read the stack trace top to bottom, find the first line pointing at your own code (not `node_modules`), and fix that.

Route Not Found / 404

Cause	Typo in the route path, wrong HTTP method, or a more general route defined earlier in the file is intercepting the request.
Identify	Falls through to Express’s default 404 handler (or your catch-all) instead of the intended controller.
Solution	Double-check the method + path match exactly, and that specific routes (`/tasks/search`) are declared before parameterized ones (`/tasks/:id`).

CORS Error (Backend Misconfiguration)

Cause	`cors()` is missing, uses the wildcard `origin: '*'` alongside `credentials: true` (invalid combination), or lists the wrong frontend domain.
Identify	Preflight `OPTIONS` request in the Network tab returns without the expected `Access-Control-Allow-*` headers.
Solution	Set an explicit origin and `credentials: true` together; never combine `'*'` with credentials (Chapter 26).

CHAPTER 43

Authentication Errors

These errors sit specifically in the login/session/OAuth path — narrower than the general back-end errors in Chapter 42, but common enough to deserve their own quick-reference.

401 Unauthorized

Cause	Request reached a protected route with no valid identity — no token, expired token, or invalid token.
Identify	Response status `401`; body typically says "Not authorized" or "No token provided."
Solution	Confirm the user is logged in and the cookie/token is actually being sent; re-authenticate if the session expired.

403 Forbidden

Cause	The user *is* authenticated but lacks the role/permission required for this action (e.g. non-admin hitting an admin route).
Identify	Response status `403`, distinct from `401` — identity is known, access is denied.
Solution	This is expected behavior for role-based routes; verify the user's role matches what `authorize(...)` requires (Chapter 21).

JWT Not Found (Cookie Missing)

Cause	The auth cookie was never set, or the browser didn't send it back with the request.
Identify	`req.cookies.token` is `undefined` in the `protect` middleware.
Solution	Confirm login actually set the cookie (check DevTools Application → Cookies) and that the frontend request includes credentials (see below).

JWT Invalid

Cause	Same root cause as Chapter 42 — secret mismatch or a tampered token — but surfaced specifically through the `protect` middleware on a login-dependent route.
Identify	`jwt.verify()` throws inside `protect`; response is `401` with "Invalid token."
Solution	Clear the bad cookie and force re-login; verify `JWT_SECRET` consistency across restarts/deploys.

JWT Expired

Cause	Token's lifetime (`expiresIn`) has elapsed since login.
Identify	`TokenExpiredError` in `protect`; user was working fine, then suddenly gets `401` on every request.
Solution	Prompt re-login on `401`; the frontend's Axios interceptor should redirect to `/login` when this happens (Chapter 33).

Cookie Not Sent

Cause	Frontend Axios instance is missing `withCredentials: true`, or the backend's `cors()` is missing `credentials: true` with a specific (non-wildcard) origin.
Identify	Cookie is visibly set in DevTools after login, but subsequent requests don't include it — `req.cookies` is empty on the backend.
Solution	Set `withCredentials: true` on the Axios instance *and* `cors({ origin: FRONTEND_URL, credentials: true })` on the backend — both sides are required.

Google OAuth: redirect_uri_mismatch

Cause	The redirect URI sent in the OAuth request doesn't exactly match one of the "Authorized redirect URIs" configured in the Google Cloud Console (protocol, domain, port, or trailing slash differs).
Identify	Google's consent screen shows "Error 400: redirect_uri_mismatch" instead of the login prompt.
Solution	Copy the exact callback URL your backend uses into the Google Cloud Console credentials, matching scheme and port precisely.

Google OAuth: invalid_client

Cause	Wrong or missing `GOOGLE_CLIENT_ID`/`GOOGLE_CLIENT_SECRET`, or credentials from the wrong Google Cloud project.
Identify	Google returns "Error 401: invalid_client" before showing the consent screen.
Solution	Re-copy the client ID/secret from the correct project into `.env` and restart the backend.

UI Not Updating After Login

Cause	Login succeeded and the cookie was set, but the React `AuthContext`'s `user` state was never refreshed, so components still render as "logged out."
Identify	Network tab shows a successful login response, but the navbar/ protected routes still behave as unauthenticated.
Solution	After a successful login call, update context state directly (or re-fetch `/auth/me`) instead of relying on a page reload (Chapter 32).

CHAPTER 44

Deployment

This chapter covers the concepts and configuration values a production MERN deployment needs, regardless of hosting provider. It does not cover Git/GitHub workflow, provider-specific CLI push mechanics, or account sign-up steps — those vary by provider and are outside this handbook's scope.

44.1 Production Architecture

Production Request Path

```
Internet
   |  HTTPS
   v
React Frontend (static hosting, e.g. CDN)
   |  HTTPS API calls (VITE_API_URL)
   v
Express Backend (Node hosting)
   |  MongoDB driver (TLS connection string)
   v
MongoDB Atlas (managed cluster)
```

Two independent deployments, connected only by URLs and environment variables — the frontend is a static build artifact, the backend is a long-running Node process, and the database is a separate managed service.

44.2 Frontend: Build for Production

```
npm run build
# produces a dist/ folder: static HTML/CSS/JS, ready for any static host
```

The static host serves `dist/` directly — no Node process runs for the frontend in production. Set `VITE_API_URL` to the deployed backend's URL *before* running `build`, since Vite inlines env variables at build time, not at runtime.

If `VITE_API_URL` still points at `localhost` when you run `build`, every API call in production will try to reach the visitor's own machine. Rebuild after changing the env value — editing `.env` alone does not update an already-built `dist/`.

44.3 Backend: Start Command

```
npm start
# package.json: "start": "node src/server.js"
```

Production runs `node`, never `nodemon` — nodemon exists purely for local file-watching and adds unnecessary overhead to a production process. The hosting platform restarts the process on crash; that is the platform's job, not nodemon's.

44.4 Environment Variables

Configure these on the hosting platform's dashboard/config system — never commit `.env` to the repository.

Variable	Production value
`MONGO_URI`	Atlas connection string for the production cluster
`JWT_SECRET`	Long random string, different from any dev secret
`NODE_ENV`	`production`
`FRONTEND_URL`	The deployed frontend's exact domain (for CORS)
`GOOGLE_CLIENT_ID/SECRET`	Production OAuth credentials with the production redirect URI registered
`CLOUDINARY_*`	Cloudinary account keys

44.5 Updating CORS for Production

```
app.use(cors({
  origin: process.env.FRONTEND_URL, // the deployed frontend's real domain
  credentials: true,
}));
```

A CORS origin of `localhost:5173` works locally but must be replaced with the deployed frontend's actual domain, or every cookie-based request will be blocked in production (Chapter 26).

44.6 MongoDB Atlas Network Access

Atlas rejects connections from IPs not on its allowlist. For a backend hosted on a platform with dynamic/unknown outbound IPs, allow `0.0.0.0/0` (any IP) — acceptable because the connection still requires valid database credentials. If the platform provides static outbound IPs, allowlist those specific addresses instead for a tighter boundary.

44.7 HTTPS and Custom Domains

Most hosting platforms provision HTTPS automatically (managed certificates) for both the frontend and backend as soon as a domain is attached — no manual certificate handling is typically required. Once HTTPS is active, set cookie options to `secure: true` and `sameSite: 'none'` for cross-domain cookie delivery (Chapter 19). A custom domain is simply a DNS record pointed at the platform, applied to either the frontend, backend, or both independently.

44.8 Debugging in Production

Every hosting platform exposes a logs view for the running backend process — this is the production equivalent of the terminal output you see locally. When something fails only in production:

- Reproduce the request and immediately check the platform's log viewer for the stack trace.
- Confirm the failing behavior isn't simply a missing/incorrect environment variable (the most common production-only bug).
- Check the browser console and Network tab on the deployed frontend exactly as you would locally.

WHAT: Two independently deployed pieces — a static frontend build and a long-running Node backend process — connected through environment variables (`VITE_API_URL`, `FRONTEND_URL`, `MONGO_URI`) and a managed MongoDB Atlas cluster.

WHY: Separating concerns lets the frontend scale as static assets on a CDN while the backend scales as a process, and keeps secrets out of the repository entirely.

HOW: Build the frontend with the production API URL baked in, start the backend with `npm start` (not nodemon), configure CORS/cookies for the real domains, and set every secret through the platform's environment variable system.

CHAPTER 45

Production Checklist

Run through this list before calling a MERN app ”done.” Each item links back to the chapter where it was implemented in detail.

45.1 Backend

- ☐ Environment variables configured on the hosting platform, none committed to the repository (Chapter 44)
- ☐ Backend production start command is `npm start` running `node src/server.js`, not `nodemon` (Chapter 44)
- ☐ CORS configured with the exact production frontend origin and `credentials: true` (Chapter 26, 44)
- ☐ Authentication tested end to end: register, login, logout, current user (Chapters 16–20)
- ☐ Authorization tested: role-restricted routes reject the wrong role with `403` (Chapter 21)
- ☐ Validation enabled on every write endpoint (Chapter 23)
- ☐ Centralized error handling middleware enabled, no raw stack traces leaked to clients (Chapter 24/36)
- ☐ Rate limiting enabled on auth and public-facing routes (Chapter 37)
- ☐ File upload size/type limits enforced on Multer config (Chapter 27, 37)
- ☐ All CRUD APIs tested with Postman/Thunder Client (Chapter 14)

45.2 Frontend

- ☐ Frontend production build (`npm run build`) run *after* setting the production `VITE_API_URL` (Chapter 44)
- ☐ API URLs updated to point at the deployed backend, no hardcoded `localhost` left in code (Chapter 41, 44)
- ☐ Mobile responsive layouts checked at `sm`/`md`/`lg` breakpoints (Chapter 35)
- ☐ Loading states shown for every async fetch, no blank screens while waiting (Chapter 33)
- ☐ Error states shown when a request fails, not a silent blank page (Chapter 33, 36)
- ☐ Empty states shown when a list legitimately has zero items, distinct from a loading or error state (Chapter 33)

45.3 Database

- ☐ MongoDB production database is a separate Atlas cluster from any dev/test database (Chapter 12, 44)
- ☐ Database connection tested against the production `MONGO_URI` before first deploy (Chapter 12, 44)
- ☐ Network Access allowlist configured (`0.0.0.0/0` or specific host IPs) (Chapter 44)

45.4 Security

- ☐ HTTPS enabled on both frontend and backend domains (Chapter 44)
- ☐ Secure cookies: `httpOnly`, `secure: true`, `sameSite` set correctly for the deployment topology (Chapter 19)
- ☐ Authentication and authorization re-tested against the *deployed* URLs, not just localhost (Chapter 20, 21, 43)

Treat this checklist as a gate, not a suggestion — a single unchecked box (most often ”CORS configured” or ”environment variables configured”) is responsible for the majority of ”it worked locally but broke in production” incidents.

CHAPTER 46

Complete Real-World Project: Task Management System

This chapter steps back and summarizes the full application built incrementally across the preceding chapters — the Task Management System, in its complete, production-ready form.

46.1 Features

Area	Capabilities
Authentication	Register, login, logout, Google OAuth login, get current user, protected routes, role-based authorization
Tasks	Full CRUD, assign to a user, status, priority, due date, search, filter, sort, pagination
User	Profile view, avatar upload, update profile
Dashboard	Total / completed / pending / in-progress counts, per-user statistics

46.2 Technical Stack

Layer	Technologies
Frontend	React, Vite, Tailwind CSS, React Router, Axios, Context API
Backend	Node.js, Express, MongoDB, Mongoose
Auth & Security	JWT, bcrypt, HttpOnly cookies, CORS, validation, error handling
Media	Cloudinary (image uploads via Multer)

46.3 Complete Project Structure

Task Management System -- Folder Tree

```
frontend/
  src/
    components/     # Button, TaskCard, Modal, Navbar, ...
    pages/          # Login, Register, Dashboard, TaskList, ...
    layouts/        # AppLayout, AuthLayout
    hooks/          # useAuth, useTasks, ...
    context/        # AuthContext
    services/       # business logic wrappers around api/
    api/            # axios instance + endpoint functions
    utils/          # formatDate, validators, ...
  package.json

backend/
```

```
src/
  config/          # db.js, cloudinary.js
  controllers/     # authController.js, taskController.js
  middleware/       # protect.js, authorize.js, errorHandler.js
  models/          # User.js, Task.js
  routes/          # authRoutes.js, taskRoutes.js
  services/        # taskService.js
  validators/      # taskValidator.js
  utils/           # generateToken.js
app.js
server.js
package.json
```

46.4 Purpose of Each Piece

Path	Purpose
components/	Small reusable UI pieces shared across pages
pages/	Route-level screens composed from components
layouts/	Shared page shells (navbar/sidebar wrapping child routes)
hooks/	Reusable stateful logic extracted out of components
context/	Global state (auth) shared without prop drilling
api/	Raw Axios calls, one function per endpoint
services/ (frontend)	Optional layer that composes/transforms api calls for pages
utils/ (frontend)	Pure helper functions (formatting, validation)
config/	Backend startup configuration (DB connection, Cloudinary)
controllers/	Request handlers – parse input, call services, send response
middleware/	Cross-cutting request logic (auth, error handling)
models/	Mongoose schemas defining data shape and rules
routes/	Maps HTTP method + path to a controller function
services/ (backend)	Business logic and database operations, controller stays thin
validators/	Input validation rule definitions
utils/ (backend)	Small backend helpers (e.g. JWT signing)
app.js	Configures Express: middleware, routes, error handler
server.js	Connects to MongoDB, then starts the HTTP server

WHAT: A single cohesive application — auth, CRUD, dashboard, file uploads — built from the individually-taught pieces in Chapters 1–45, organized into the layered frontend and backend structures introduced early in the book.

WHY: Seeing the whole project assembled makes the relationships between layers (route → controller → service → model, or page → hook → api) concrete instead of abstract.

HOW: Use this structure as a template: any new MERN project can start from this same folder layout and feature set, then grow from there.

CHAPTER 47

End-to-End Implementation Order

This is the order to actually build a new MERN project, start to finish. Each step references the chapter that covers it in depth.

1. **Create frontend** (Vite + React) — Chapter 2, 3
2. **Create backend** (Node + Express project) — Chapter 2
3. **Configure Express** (`app.js`, middleware, `server.js`) — Chapter 9
4. **Connect MongoDB** (`mongoose.connect` in `server.js`) — Chapter 12
5. **Create User model** (Mongoose schema) — Chapter 13
6. **Create Task model** (Mongoose schema, relations) — Chapter 13, 15
7. **Create auth routes** (`/register`, `/login`, `/logout`, `/me`) — Chapter 10, 16
8. **Create auth controllers** (register/login/logout logic) — Chapter 16
9. **Implement bcrypt** (hash on save, compare on login) — Chapter 17
10. **Implement JWT** (sign a token on successful login) — Chapter 18
11. **Implement cookies** (send the JWT as an HttpOnly cookie) — Chapter 19
12. **Create auth middleware** (`protect`, `authorize`) — Chapter 20, 21
13. **Create task APIs** (routes for the Task resource) — Chapter 10, 14
14. **Create CRUD** (controllers + service layer for tasks) — Chapter 11, 14
15. **Create React routes** (React Router setup, protected routes) — Chapter 5
16. **Create AuthContext** (global auth state on the frontend) — Chapter 32
17. **Create Axios instance** (baseURL, `withCredentials`, interceptors) — Chapter 7
18. **Connect APIs** (wire pages to the api/ layer end to end) — Chapter 8
19. **Build forms** (login, register, task create/edit) — Chapter 34
20. **Build dashboard** (stats, counts, aggregation queries) — Chapter 31, 46
21. **Add search/filter** (query params on the list endpoint) — Chapter 28
22. **Add pagination** (skip/limit or cursor-based) — Chapter 29
23. **Add image upload** (Multer + Cloudinary) — Chapter 27
24. **Add validation** (backend request validation) — Chapter 23
25. **Add security** (rate limiting, headers, sanitization) — Chapter 37
26. **Test with Postman** (every endpoint, happy path + errors) — Chapter 14, 25
27. **Test frontend** (manual pass through every screen and state) — Chapter 33, 36

28. **Configure production** (env vars, CORS origin, cookie flags) — Chapter 44
29. **Deploy** (build frontend, start backend, connect Atlas) — Chapter 44
30. **Test production** (repeat the Postman/frontend pass against live URLs) — Chapter 45

Steps 1–17 form the ”skeleton” — once auth and one full CRUD resource work end to end, every additional feature (search, pagination, uploads) is an incremental addition to an already-working request/response loop, not a new architecture.

CHAPTER 48

Key Architecture Diagrams

Five diagrams that summarize the flows built throughout this book. Refer back here whenever you need the big picture without re-reading a full chapter.

1. MERN Architecture

```
React --Axios--> Express --> Controller --> Service --> Mongoose --> MongoDB
```

Every request flows through the same five layers, in the same order, regardless of which feature it belongs to.

2. Authentication Flow

```
Login --> Verify Password --> Generate JWT --> Set Cookie --> Browser

Browser --Protected Request--> Middleware --> JWT Verify --> Controller
```

Login issues the token once; every subsequent protected request re-proves identity by having the middleware verify that same token.

3. CRUD Flow

```
React Form --POST--> Express --> Controller --> MongoDB
                                                    |
React State <-- UI Update <-- Response <-------------
```

The round trip always ends by updating local React state so the UI reflects the new database state without a full page reload.

4. File Upload Flow

```
React --FormData--> Multer --> Cloudinary --> URL --> MongoDB (saved)
```

The file itself never touches MongoDB — only the Cloudinary-hosted URL is persisted on the document.

5. Production Deployment Flow

```
User --> Frontend Hosting --HTTPS--> Backend API --> MongoDB Atlas
```

Three independently deployed pieces, connected only by HTTPS URLs and environment variables — never by shared filesystem or process.

CHAPTER 49

MERN Quick Reference

Dense cheat tables — method name, one-line purpose. No explanations beyond this; see the referenced chapters for detail.

49.1 Express

Method	Purpose
`app.use()`	Register middleware/router for all or a prefixed path
`app.get()`	Handle GET requests on a path
`app.post()`	Handle POST requests on a path
`app.put()`	Handle PUT (full replace) requests
`app.patch()`	Handle PATCH (partial update) requests
`app.delete()`	Handle DELETE requests
`Router()`	Create a modular, mountable route handler
`req.params`	Named route segments (`/tasks/:id`)
`req.query`	URL query string values (`?status=done`)
`req.body`	Parsed request body (needs `express.json()`)
`req.headers`	Raw request headers
`res.json()`	Send a JSON response body
`res.status()`	Set the HTTP status code before sending

49.2 Mongoose

Method	Purpose
`Schema`	Define document shape, types, and validation rules
`model()`	Compile a Schema into a usable Model
`find()`	Query multiple documents matching a filter
`findOne()`	Query the first document matching a filter
`findById()`	Query a document by its `_id`
`create()`	Insert a new document
`save()`	Persist changes on a document instance
`updateOne()`	Update the first matching document, no hooks
`findByIdAndUpdate()`	Find by id and update in one call
`deleteOne()`	Delete the first matching document
`findByIdAndDelete()`	Find by id and delete in one call
`populate()`	Replace a referenced `_id` with the full document
`aggregate()`	Run a multi-stage data pipeline
`sort()`	Order query results
`limit()`	Cap the number of returned documents
`skip()`	Offset results (used with `limit` for pagination)

49.3 React

Hook	Purpose
`useState`	Local component state
`useEffect`	Run side effects after render (fetches, subscriptions)
`useContext`	Read a value from a Context Provider
`useRef`	Persist a mutable value without triggering re-renders
`useMemo`	Cache an expensive computed value between renders
`useCallback`	Cache a function reference between renders

49.4 Axios

Call	Purpose
`api.get()`	Fetch data
`api.post()`	Create a resource
`api.put()`	Replace a resource
`api.patch()`	Partially update a resource
`api.delete()`	Remove a resource
`interceptors`	Run logic before every request/response (e.g. attach headers, handle 401)
`baseURL`	Shared root URL for all calls from one instance
`withCredentials`	Include cookies on cross-origin requests

49.5 Authentication

Concept	Purpose
`bcrypt`	One-way password hashing before storage
`JWT`	Signed token proving identity without a server-side session store
`cookie`	Transport mechanism for delivering the JWT to the browser
`HttpOnly`	Cookie flag blocking JavaScript access (XSS mitigation)
`Secure`	Cookie flag restricting transmission to HTTPS
`SameSite`	Cookie flag controlling cross-site sending behavior
`middleware`	Function that runs before the controller (e.g. `protect`)
`authorization`	Role/permission check run after authentication succeeds

Final Revision: MERN Project-Building Cheat Sheet

Rapid-reference recap of the whole book. Terse by design — if a line doesn't make sense, the full explanation is in the chapter it summarizes.

Page 1: MERN Architecture + Folder Structure

Layered Architecture

```
Browser (React)
   | Axios (HTTPS)
Express App (app.js)
   | Router -> Middleware -> Controller
Service Layer (business logic)
   | Mongoose
MongoDB Atlas
```

Combined Project Tree

```
frontend/src/{components,pages,layouts,hooks,context,api,utils}
backend/src/{config,controllers,middleware,models,routes,
             services,validators,utils}
backend/{app.js,server.js}
```

Page 2: Frontend to Backend API Communication

Full Request/Response Chain

```
React event -> api.get('/tasks') -> Express route -> middleware(s)
  -> controller -> service -> Mongoose -> MongoDB
MongoDB -> Mongoose -> service -> controller -> res.json()
  -> Axios response -> setState -> re-render
```

```
// frontend: src/api/tasks.js
export const getTasks = () => api.get("/tasks");

// backend: routes/taskRoutes.js
router.get("/tasks", protect, getTasks);

// backend: controllers/taskController.js
const getTasks = async (req, res) => {
  const tasks = await Task.find({ user: req.user._id });
  res.status(200).json(tasks);
};
```

Page 3: Express + MongoDB + Mongoose

Method	Purpose
`app.use/get/post/put/patch/delete`	Middleware + route handlers per HTTP verb
`req.params/query/body`	Route params / query string / parsed body
`res.status().json()`	Send status code + JSON body
`Schema / model()`	Define shape + compile to usable Model
`find/findOne/findById`	Read one or many documents
`create/save`	Insert or persist a document
`findByIdAndUpdate/Delete`	Update or delete by id in one call
`populate/sort/limit/skip`	Join refs, order, and paginate results

```
// config/db.js
const connectDB = async () => {
  await mongoose.connect(process.env.MONGO_URI);
  console.log("MongoDB connected");
};

// app.js
const app = express();
app.use(express.json());
app.use(cors({ origin: process.env.FRONTEND_URL, credentials: true }));
app.use("/api/tasks", taskRoutes);
app.use(errorHandler);

// server.js
connectDB().then(() =>
  app.listen(process.env.PORT, () => console.log("Server running"))
);
```

Page 4: Authentication + JWT + Cookies + OAuth

Auth Flow

```
Register -> bcrypt.hash -> save user
Login -> bcrypt.compare -> jwt.sign -> Set-Cookie (HttpOnly)
Protected request -> cookie sent automatically -> protect middleware
  -> jwt.verify -> req.user set -> controller
```

```
// register/login (controller)
const hashed = await bcrypt.hash(password, 10);
const match = await bcrypt.compare(password, user.password);
const token = jwt.sign({ id: user._id }, process.env.JWT_SECRET, {
  expiresIn: "7d",
});
res.cookie("token", token, {
  httpOnly: true, secure: true, sameSite: "none",
});

// middleware/protect.js
const protect = async (req, res, next) => {
  const token = req.cookies.token;
  if (!token) return res.status(401).json({ message: "Not authorized" });
  const decoded = jwt.verify(token, process.env.JWT_SECRET);
  req.user = await User.findById(decoded.id);
  next();
};
```

Cookie Option	Meaning
`httpOnly`	Blocks JS access to the cookie (XSS mitigation)
`secure`	HTTPS-only transmission
`sameSite`	`none` (cross-domain, needs `secure`) or `lax` (same-site default)

Google OAuth in one line: frontend redirects to Google → Google redirects back to backend's callback URL with a code → backend exchanges the code for profile info → backend creates/finds the user and issues its own JWT cookie exactly like a normal login.

Page 5: Errors, Debugging, Deployment, Production Checklist

Debugging order: Console error → Network tab (status + response) → backend terminal stack trace → fix the first line pointing at your own code.

Top Frontend Errors	Fix
CORS error	Backend `cors()` needs matching origin + credentials
Network Error	Backend down / wrong URL / wrong port
404 on API call	Path typo or route not mounted
Cannot read '...' of undefined	Guard render until data has loaded
Env var undefined	Missing `VITE_` prefix or dev server not restarted

Top Backend Errors	Fix
EADDRINUSE	Kill the process on that port or change `PORT`
Mongo connection failed	Check URI, credentials, Atlas IP allowlist
req.body undefined	Add `express.json()` before routes
req.user undefined	Add `protect` middleware to the route
E11000 duplicate key	Unique index violation, handle in controller

Deployment Flow

```
User -> Frontend Hosting -HTTPS-> Backend API -> MongoDB Atlas
```

Production checklist (condensed):

- ☐ Env vars set on platform, none committed
- ☐ CORS origin = real frontend domain, HTTPS enabled
- ☐ Secure cookies (`httpOnly`, `secure`, `sameSite`)
- ☐ Auth + authorization tested against deployed URLs
- ☐ Validation, error handling, rate limiting enabled
- ☐ Frontend built with production API URL; backend started with `npm start`
- ☐ All CRUD tested; loading/error/empty states present
- ☐ Mobile responsive checked